WOJSKOWA AKADEMIA TECHNICZNA

im. Jarosława Dąbrowskiego

WYDZIAŁ CYBERNETYKI



PRACA DYPLOMOWA

STUDIA I°

Temat pracy: **ŚRODOWISKO WIZUALIZACJI SYMULACJI PROCESÓW BIZNESOWYCH**

INFORMATYKA

(kierunek studiów)

INŻYNIERIA SYSTEMÓW

(specjalność)

Dyplomant:

Wojciech KIELAK

Promotor:

dr inż. Adrian WOŹNIAK

Warszawa 2024

OŚWIADCZENIE

Wyrażam zgodę / ~~nie wyrażam~~ ~~zgody~~*
na udostępnianie mojej pracy w czytelni Archiwum WAT.*

Dnia 25.01.2024 Wojciech Kielak
(podpis)

**Niepotrzebne skreślić*

Spis treści

Wstęp 4
Rozdział I. Opis zagadnień związanych z symulacją procesów biznesowych 5
I.1. Proces biznesowy 5
I.2. Funkcjonowanie symulacji 10
I.3. Wymagania funkcjonalne 12
I.4. Wymagania pozafunkcjonalne 14
Rozdział II. Struktura aplikacji i integracji 15
II.1. Architektura aplikacji 15
II.2. Model danych 18
Rozdział III. Opis implementacji 22
III.1. Ekran wyporu organizacji 22
III.2. Ekran symulacji 24
III.3. Ekran raportu 35
Rozdział IV. Testy 37
Podsumowanie 47
Bibliografia 48
Spis kodów 49
Spis rysunków 49
Spis tabel 50
Załączniki 50

Wstęp

W dobie XXI wieku gospodarka rozwija się dynamicznie, rynek charakteryzuje się nieprzewidywalnością oraz szybką zmianą. Wymusza to na przedsiębiorstwach nieustanne dostosowywanie się do nowej rzeczywistości, a zrozumienie i efektywne zarządzanie procesami biznesowymi staje się kluczowe dla utrzymania innowacyjności i konkurencyjności. Wielu przedsiębiorców boryka się z wyzwaniem identyfikacji słabych rozwiązań stosowanych w ramach swojej organizacji, co w dzisiejszych czasach staje się coraz trudniejszym przedsięwzięciem. Symulacja procesów biznesowych może pokazać, gdzie znajduje się problem, a ponad to pomoże odpowiedzieć na pytanie co trzeba poprawić, aby biznes działał prawidłowo. Na rynku istnieje wiele aplikacji, które umożliwiają symulację procesów biznesowych, ale niewiele jest aplikacji internetowych tego rodzaju.

Celem pracy jest przedstawienie środowiska wizualizacji symulacji procesów biznesowych. Aplikacja ta może nie tylko ułatwić zrozumienie struktur i zależności wewnętrznych organizacji, ale również umożliwi przewidywanie efektów wprowadzanych zmian, innowacji i ujawni błędy w funkcjonowaniu firmy.

Rozdział pierwszy pracy poświęcony jest teoretycznym podstawom symulacji procesów biznesowych, definiując kluczowe pojęcia i prezentując istotę symulacji w kontekście funkcjonowania przedsiębiorstw. Kolejne podrozdziały przedstawiają problematykę wymagań funkcjonalnych i poza funkcjonalnych. Drugi rozdział przedstawia strukturę aplikacji i użyte biblioteki. Omówiona została również struktura przetwarzanych danych, na których opiera się symulacja. Trzeci rozdział skupia się na opisie implementacji aplikacji, w której omówione są interfejsy użytkownika. Znajduje się w nim omówienie interakcji użytkownika z aplikacją. Ostatni rozdział poświęcony jest testom aplikacji. Pozwoliły one na weryfikacje poprawności działania aplikacji.

Rozdział I. Opis zagadnień związanych z symulacją procesów biznesowych

I.1. Proces biznesowy

Według autorów [1] proces jest zbiorem działań, zdarzeń i decyzji, które prowadzą do wyniku przynoszącego wartość klientom organizacji. Każda firma posiada procesy. Zrozumienie i zarządzanie procesami w celu zapewnienia, że wytwarzają one wartość, może być kluczowym składnikiem konkurencyjności i skuteczności organizacji.

Procesy biznesowe stanowią podstawowy zasób firm i mają bezpośredni wpływ na atrakcyjność produktów i usług postrzeganych na rynku. Określają miejsca pracy, zadania, obowiązki i tym kształtują pracę każdego pracownika. Procesy mogą mieć wpływ na przychody, a tym samym oddziałują na koszty organizacji. Jednak w przeciwieństwie do innych aktywów spółki, takich jak produkty, marka, usługi, siła robocza, aktywa pieniężne i rzeczowe, znaczenie procesów biznesowych nie było przez długi czas doceniane. Dopiero rosnące wymagania dotyczące globalizacji, standaryzacji, innowacji, integracji, zwinności i efektywności operacyjnej oraz związane z tym wyzwanie znalezienia kolejnych zmiennych w ekosystemie korporacyjnym, które można poprawić, zwiększyły apetyt na refleksję i usprawnienie procesów biznesowych. W odpowiedzi na tę potrzebę, opracowano kompleksowy zestaw technik, narzędzi, metod i metodologii wspierających modelowanie procesów biznesowych.

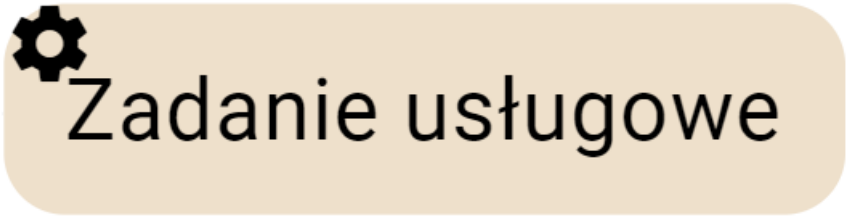
BPM (z ang. Buissnes Process Model) - zarządzanie procesami biznesowymi jest to zbiór metod, narzędzi i technik do odkrywania, analizowania, wykonywania, przeprojektowywania i monitorowania procesów biznesowych. Kluczowym założeniem BPM jest koncentracja na procesach jako podstawie do zarządzania i organizacji pracy w przedsiębiorstwie.

Przed rozpoczęciem modelowania procesu kluczowe jest zrozumienie potrzeb i celów jego modelowania. Wytwarzane modele, będą wyglądać zupełnie inaczej w zależności od powodu ich modelowania. Pierwszym krokiem jest zrozumienie procesu i skonfrontowanie zrozumienia procesu z ludźmi, którzy są zaangażowani w proces na co dzień. Modelowanie procesu umożliwia lepsze jego zrozumienie i identyfikację potencjalnych problemów. Jest to fundament do dalszej analizy czy automatyzacji procesów.

BPMN (z ang. Buissnes Process Model and Notation) - Notacja Procesów Biznesowych to powszechnie stosowany standard modelowania procesów biznesowych. Głównym celem BPMN jest dostarczenie notacji, która jest łatwo zrozumiała dla szerokiego spektrum użytkowników - od analityków biznesowych, którzy tworzą wstępne projekty procesów, po programistów technicznych odpowiedzialnych za wdrażanie technologii, która będzie je wykonywać i biznesmenów, którzy będą nimi zarządzać i monitorować. Podstawowe pojęcia BPMN to zdarzenie, czynność i łuk. Zdarzenia są reprezentowane przez okręgi, czynności przez

zaokrąglone prostokąty, a łuki nazywane też przepływami sekwencji są oznaczane przez strzałki z wypełnionym grotem.

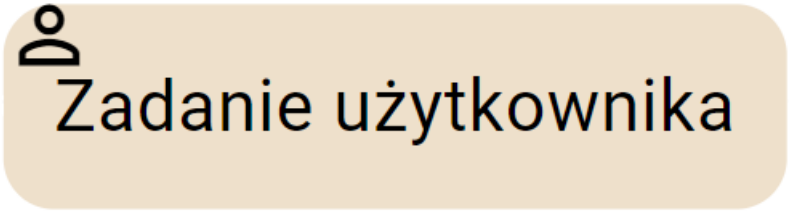
Czynności to pewien fragment pracy realizowanych w procesie biznesowym. Mogą być nimi zadania, czyli takie czynności, których nie dzieli się na mniejsze. Zadania abstrakcyjne są reprezentowane przez prostokąty z zaokrąglonymi rogami bez żadnych dodatkowych symboli. Używa ich się do modelowania procesu bez precyzowania typu zadań. Zadanie usługowe to takie, które jest realizowanie całkowicie automatycznie, bez udziału człowieka. Jest ono reprezentowane przez prostokąt z zaokrąglonymi rogami oraz ikoną usługi. Reprezentacja ta została przedstawiona na rysunku 1.



Rys. 1. Zadanie usługowe

Źródło: opracowanie własne

Zadanie użytkownika jest realizowane przez człowieka, z wykorzystaniem aplikacji komputerowej. Na rysunku 2 przedstawione zostało zadanie użytkownika.



Rys. 2. Zadanie użytkownika

Źródło: opracowanie własne

Zadanie manualne jest realizowanie przez człowieka, ale całkowicie poza systemami oprogramowania. Przykładem takiego zadania manualnego może być podpisywanie dokumentów. Zadanie manualne zostało przedstawione na rysunku 3.



Rys. 3. Zadanie manualne

Źródło: opracowanie własne

Zdarzenia i czynności mogą nie być wykonywane po kolei, przykładowo w ramach procesu, w którym występują dwie wzajemnie wykluczające się czynności. Zatem czynności te nie mogą być wykonywane po kolei, ponieważ jeden z etapów tego procesu będzie dotyczył obu tych czynności. Do opisania zachowań, gdzie dwie lub więcej czynności są dla siebie alternatywne, czyli wzajemnie się wykluczają, używa się bramki.

Bramka oznacza, że istnieje mechanizm pośredniczący, który albo pozwala na przejście procesu przez bramkę lub nie. W miarę docierania elementów do bramy mogą one być łączone razem na wejściu lub rozdzielane na wyjściu, w zależności od typu bramki. Podstawowymi typami bramek są: bramka XOR, bramka OR i bramka AND.

Według autora [3] bramka XOR jest wtedy, gdy w procesie następują warunkowe rozgałęzienia, czyli istnieje kilka alternatywnych ścieżek, ale tylko jedna z nich może zostać wybrana w danym przebiegu. Jest stosowana do modelowania relacji między dwoma lub więcej alternatywnymi działaniami, jak na przykład zatwierdzenie lub odrzucenie wniosku, a także do ponownego połączenia gałęzi rozdzielonych wcześniej. Bramka XOR jest przedstawiana znakiem diamentu, opcjonalnie ze znakiem „X" wewnątrz niego. Symbol bramki XOR został pokazany na rysunku 4.



Rys. 4. Bramka XOR

Źródło: opracowanie własne

Bramka AND jest stosowana, gdy w procesie czynności są realizowane niezależnie, czyli jedna czynność nie musi następować po drugiej ani nie wyklucza drugiej. Jest ona reprezentowana za pomocą znaku plusa wewnątrz diamentu. Umożliwia jednoczesne przetwarzanie wielu ścieżek, czyli dwa równoległe procesy nie muszą być realizowane w tej samej chwili, ale mogą. Natomiast na pewno muszą być wzajemnie niezależne. Symbol bramki AND został przedstawiony na rysunku 5.

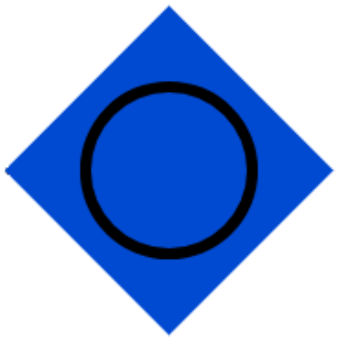


Rys. 5. Bramka AND

Źródło: opracowanie własne

Bramka OR ma zastosowanie w przypadkach, gdy w procesie występują rozgałęzienia, czyli istnieje kilka alternatywnych innych ścieżek i możliwe jest wybranie kilku z nich.

Warunki na jego wychodzących gałęziach nie muszą się wzajemnie wykluczać, czyli więcej niż jeden z nich może być prawdziwy w tym samym czasie. Bramka ta jest oznaczona poprzez znak „O” wewnątrz diamentu. Na rysunku 6 został przedstawiony symbol bramki OR.



Rys. 6. Bramka OR

Źródło: opracowanie własne

Czynności realizowane są przez zasoby, w kontekście zarządzania nimi w procesach biznesowych, istotne jest zrozumienie roli różnych typów zasobów. Zasób wskazuje, kto lub co wykonuje konkretną czynność, jest to ogólny termin odnoszący się do kogokolwiek lub czegokolwiek zaangażowanego w wykonywanie czynności procesu. Rozróżniane są zasoby aktywne, czyli takie, które mogą autonomicznie wykonywać czynność, oraz zasoby pasywne, które służą jako wsparcie w realizacji zadania. Na przykład drukarka jest używana przez uczestnika do drukowania dokumentów, ale to uczestnik wykonuje czynność drukowania. Drukarka jest więc zasobem pasywnym, podczas gdy uczestnik jest zasobem aktywnym. Dalej, koparka jest kolejnym przykładem zasobu pasywnego, ponieważ to kierowca wykonuje czynność, w której koparka jest używana.

BPMN udostępnia dwie konstrukcje do modelowania aspektów zasobów: baseny (z ang. pool) i tory pływackie (z ang. lane), zwane również pasami. Baseny są zwykle używane do modelowania klas zasobów, a tory pływackie są używane do podziału basenów na podklasy lub pojedyncze zasoby. Nie ma żadnych ograniczeń co do tego, jaki konkretny typ zasobu powinien modelować basen lub tor pływacki. Zazwyczaj basenu używa się do modelowania strony biznesowej, takiej jak cała organizacja, a toru pływackiego do modelowania działu, jednostki, zespołu lub systemu oprogramowania w ramach firmy. Baseny i tory pływackie są przedstawiane jako prostokąty, w których można umieszczać czynności, zdarzenia i bramy. Zazwyczaj modeluje się te prostokąty poziomo, choć możliwe jest również modelowanie ich pionowo. Nazwa basenu lub toru pływackiego jest wyświetlana pionowo po lewej stronie poziomego prostokąta lub poziomo, jeśli basen i tor pływacki jest pionowy.

Podstawowy model BPMN obejmuje proste działania, bramy, zdarzenia, baseny i tory pływackie. Działania przechwytują jednostki pracy w ramach procesu. Zdarzenia definiują początek i koniec procesu oraz sygnalizują co dzieje się podczas jego

wykonywania. Bramy modelują łączenie, równoległość i synchronizację oraz powtarzanie. Model procesu przedstawia możliwe sposoby wykonania danego procesu biznesowego, podczas gdy instancja procesu przechwytuje jedno konkretne wykonanie procesu spośród wszystkich możliwych. Baseny generalnie modelują klasy zasobów, podczas gdy tory pływackie są używane do dalszego podziału basenu.

Zdarzenia są używane do modelowania czegoś, co dzieje się natychmiast w procesie. Zdarzenie startu sygnalizuje, uruchomienie instancji procesu oraz zdarzenie zakończenia, które sygnalizują zakończenie instancji procesu.

Wielu przedsiębiorców chciałoby, aby procesy były tańsze, szybsze i lepsze. Ta prosta obserwacja prowadzi do zidentyfikowania trzech podstawowych wymiarów wydajności procesów: czasu, kosztów i jakości Dodatkowym czynnikiem jest elastyczność, która jest czasem brana pod uwagę w kontekście adaptacji do zmieniających się warunków.

Czas jest jednym z głównym kryteriów efektywności procesów biznesowych. Inne aspekty wymiaru czasu pojawiają się, po uwzględnieniu składowych procesu. Czas przetwarzania nazwany również czasem obsługi jest to czas jaki zasoby poświęcają na faktyczną obsługę sprawy. Czas oczekiwania to czas, w którym sprawa spędza w trybie bezczynności. Czas oczekiwania obejmuje okres w kolejce, które to czekanie spowodowane jest tym, że nie są dostępne żadne zasoby do obsługi sprawy. Może też być spowodowany oczekiwaniem na synchronizację z innym procesem lub na informacje wejściowe od klienta, czy innego zewnętrznego aktora.

Koszt jest kolejnym powszechnym wymiarem wydajności podczas analizy procesu biznesowego, ma charakter finansowy. Istnieją różne perspektywy dotyczące kosztów. Jedną z nich jest rozróżnienie kosztów zmiennych i stałych. Koszty stałe to koszty ogólne, które praktycznie nie zależą od intensywności przetwarzania. Typowe koszty stałe wynikają z utrzymania systemów informatycznych i wykorzystania infrastruktury. Koszt zmienny jest związany z poziomem sprzedaży, liczbą zakupionych towarów i liczbą nowych pracowników. Koszt operacyjny jest ściśle związany z produktywnością. Koszty operacyjne mogą być związane z wynikami procesu biznesowego. Znaczną częścią kosztów operacyjnych są zazwyczaj koszty pracy, czyli takie koszty, które są związane z zasobami ludzkimi przy produkcji towarów czy świadczeniu usług. W ramach działań związanych z przeprojektowywaniem procesów bardzo często celem staje się redukcja kosztów operacyjnych, a w szczególności kosztów pracy. Alternatywą dla siły roboczej coraz częściej staje się automatyzacja zadań. Dzieje się tak, ponieważ automatyzacja może obniżyć koszty pracy, ale może ona również powodować dodatkowe koszty związane z opracowaniem odpowiedniego oprogramowania i stałymi kosztami utrzymania jej przez cały okres użytkowania aplikacji.

I.2. Funkcjonowanie symulacji

Podstawowa idea symulacji procesów biznesowych jest dość prosta. Zasadniczo, symulator procesu generuje dużą liczbę hipotetycznych instancji, które wykonują się krok po kroku. Każde działanie jest rejestrowane w symulatorze. Informacje wyjściowe symulatora zazwyczaj zawierają dzienniki symulacji i statystyki związane z średnim czasem oczekiwania, czasem cyklu i średnim wykorzystaniem zasobów.

Symulacja procesu biznesowego pozwala na modelowanie i ocenę wydajności procesów. Podczas symulacji procesu zadania nie są faktycznie wykonywane w procesie. Zamiast tego symulacja przebiega w następujący sposób. Po wygenerowaniu nowego zadania tworzony jest tak zwany element roboczy. W symulatorze najpierw szukany jest zasób, który mógłby obsłużyć element roboczy. Jeśli nie zostanie znaleziony żaden wolny zasób zdolny do wykonania elementu, to element roboczy jest umieszczany do kolejki w trybie oczekiwania, aż do momentu, gdy zostanie zwolniony zasób zdolny do obsługi zadania. Po przypisaniu zasobu do elementu roboczego, w symulatorze określany jest czas trwania elementu roboczego. Czas oczekiwania ustalany jest przez wylosowanie liczby losowej zgodnie z rozkładem prawdopodobieństwa czasu przetwarzania zadania. Rozkład prawdopodobieństwa i odpowiadające im parametry muszą być zdefiniowane w modelu symulacyjnym. Po określeniu czasu trwania elementu roboczego, następuje przełączenie go w tryb uśpienia Tryb uśpienia symuluje fakt, że zadanie jest wykonywane. Po wyjściu z trybu uśpienia, element roboczy zostaje uznany za ukończony i zasób, który został do niego przypisany, staje się ponownie dostępny. W symulatorze używa się inteligentnych algorytmów, aby ukończyć symulację tak szybko, jak to możliwe. Dla każdego elementu pracy utworzonego podczas symulacji, rejestrowany jest identyfikator zasobu, który został przypisany instancji, oraz trzy znaczniki czasu. Pierwszym znacznikiem jest czas, w którym zadanie było gotowe do wykonania. Drugim jest czas, w którym zadanie zostało uruchomione, co oznacza, że zostało przypisane do zasobu. Trzecim znacznikiem jest czas zakończenia zadania. Korzystając z zebranych danych, można obliczyć średni czas oczekiwania dla każdego zadania. Symulator rejestruje, jakie zasoby wykonują poszczególne elementy pracy i w jakim czasie. Na podstawie czasu zajętości zasobu podczas symulacji i całkowitego czasu trwania symulacji, można wyliczyć wykorzystanie zasobu, czyli procent czasu, w którym zasób jest zajęty.

W procesach biznesowych, w przypadkach, gdy zasoby są ograniczone może wystąpić rywalizacja o nie, która może prowadzić do powstania kolejek. Teoria kolejek to zbiór technik matematycznych służących do analizy systemów, w których występuje rywalizacja o zasoby. Dostarcza ona narzędzia umożliwiające badanie parametrów kolejki, takich jak oczekiwana długość kolejki lub czas oczekiwania. W ramach modelu symulacji dla każdego zadania należy określić zasób, zdolny do wykonania czynności oraz rozkład prawdopodobieństwa dla czasu przetwarzania każdego zadania. Jednym z typowych rozkładów prawdopodobieństwa dla czasu trwania zadań w kontekście

symulacji procesu jest rozkład stały. Jest to przypadek, w którym czas wykonywania zadania przez zasób jest taki sam za każdym razem. Jednak zadania z rozkładem stałym nie są częste, ponieważ większość zadań wykazuje pewną zmienność czasu przetwarzania, zwłaszcza czynności, w których udział biorą zasoby ludzkie. Rozkład wykładniczy ma zastosowanie, gdy czas przetwarzania zadania oscyluje wokół wartości średniej, ale czasami bywa znacznie dłuższy. Oznacza to, że rzeczywisty czas przetwarzania może być większy lub mniejszy od średniej z takim samym prawdopodobieństwem. Przypisując rozkład wykładniczy do czasu trwania zadania, należy określić wartość średnią. Tymczasem podczas przypisywania rozkładu normalnego należy określić dwa parametry takie jak: wartość średnią i odchylenie standardowe. Wartości parametrów rozkładów uzyskuje się poprzez świadome zgadywanie lub poprzez analizę dzienników systemów informatycznych. Bądź też za pomocą zbierania próbek dla wykonywanych zadań. Oprócz danych symulacyjnych dla każdego zadania w procesie, należy jeszcze określić prawdopodobieństwo rozgałęzienia dla każdego łuku wychodzącego z bramki decyzyjnej. Prawdopodobieństwa są określane się poprzez rozmowy z odpowiednimi interesariuszami oraz przez obserwację wykonania zadania w procesie.

Przed uruchomieniem symulacji należy jeszcze określić czas między przybyciem, jest to czas jaki upływa między pojawianiem się nowo generowanych hipotetycznych instancji w procesie. Częstym rozkładem czasu między przybyciem jest rozkład wykładniczy i jest to zwykle domyślny rozkład dla symulacji procesów biznesowych. Może się jednak zdarzyć, że czas między przybyciem ma inny rozkład niż wykładniczy, na przykład rozkład normalny. Analiza statystyczna próbki czasu między przybyciem może pomóc w ustaleniu, który rozkład wystarczająco odpowiada rzeczywistości. W modelu symulacji zazwyczaj istnieje kilka opcji określenia czasu trwania symulacji. Można wybrać datę i godzinę rozpoczęcia oraz datę i godzinę zakończenia symulacji, umożliwia to określenie czasu trwania symulacji. Alternatywnie, może istnieć możliwość podania czasu trwania symulacji bez konkretnych dat i godzin. Jeśli opcja czasu symulacji zostanie wybrana, to podczas symulacji generowanie nowych instancji procesu zakończy się, gdy zegar symulacji osiągnie czas zakończenia. Kolejną opcją jest liczba instancji procesu do symulacji. Po wybraniu opcji z liczbą instancji procesu, w symulatorze generowane są instancje procesu zgodnie z szybkością przybycia, aż do momentu osiągnięcia wymaganej liczby instancji procesu. W tym momencie symulacja procesów biznesowych może zostać zatrzymana. Niektóre symulatory nie zatrzymują się od razu, ale pozwalają na zakończenie rozpoczętych instancji procesu przed zakończeniem symulacji. Według autorów [2] analiza przepływu pozwala na obliczenie wydajności na podstawie modelu procesu biznesowego i danych dotyczących wydajności odnoszących się do każdego działania w modelu. Analiza przepływu nie uwzględnia poziomu zajętości zasobów zaangażowanych w proces, czyli poziomu ich wykorzystania. Jednak czas oczekiwania

jest w dużej części zależny od wykorzystania zasobów. Im bardziej zajęte są zasoby, tym dłuższe są czasy oczekiwania. Podstawowe modele teorii kolejek, umożliwiają obliczenie czasu oczekiwania dla poszczególnych działań. Uwzględniając informacje o liczbie zasobów i czasach ich przetwarzania. Symulacja procesu pozwala uzyskać miary wydajności procesu, biorąc pod uwagę informacje o działaniach i o zasobach zaangażowanych w proces.

Symulacja procesów biznesowych umożliwia obliczenie średniego czasu oczekiwania dla każdego zadania w procesie biznesowym. Średni czas oczekiwania może okazać się pomocny w identyfikacji zatorów, czyli wąskich gardeł w procesie. Jeśli zadanie w procesie biznesowym ma wysoki średni czas oczekiwania, może to oznaczać wystąpienie wąskiego gardła. Informacje uzyskane podczas symulacji mogą zostać również wykorzystane do przewidywania przyszłych czasów realizacji procesów, co może umożliwić ich monitorowanie i efektywne zarządzanie.

I.3. Wymagania funkcjonalne

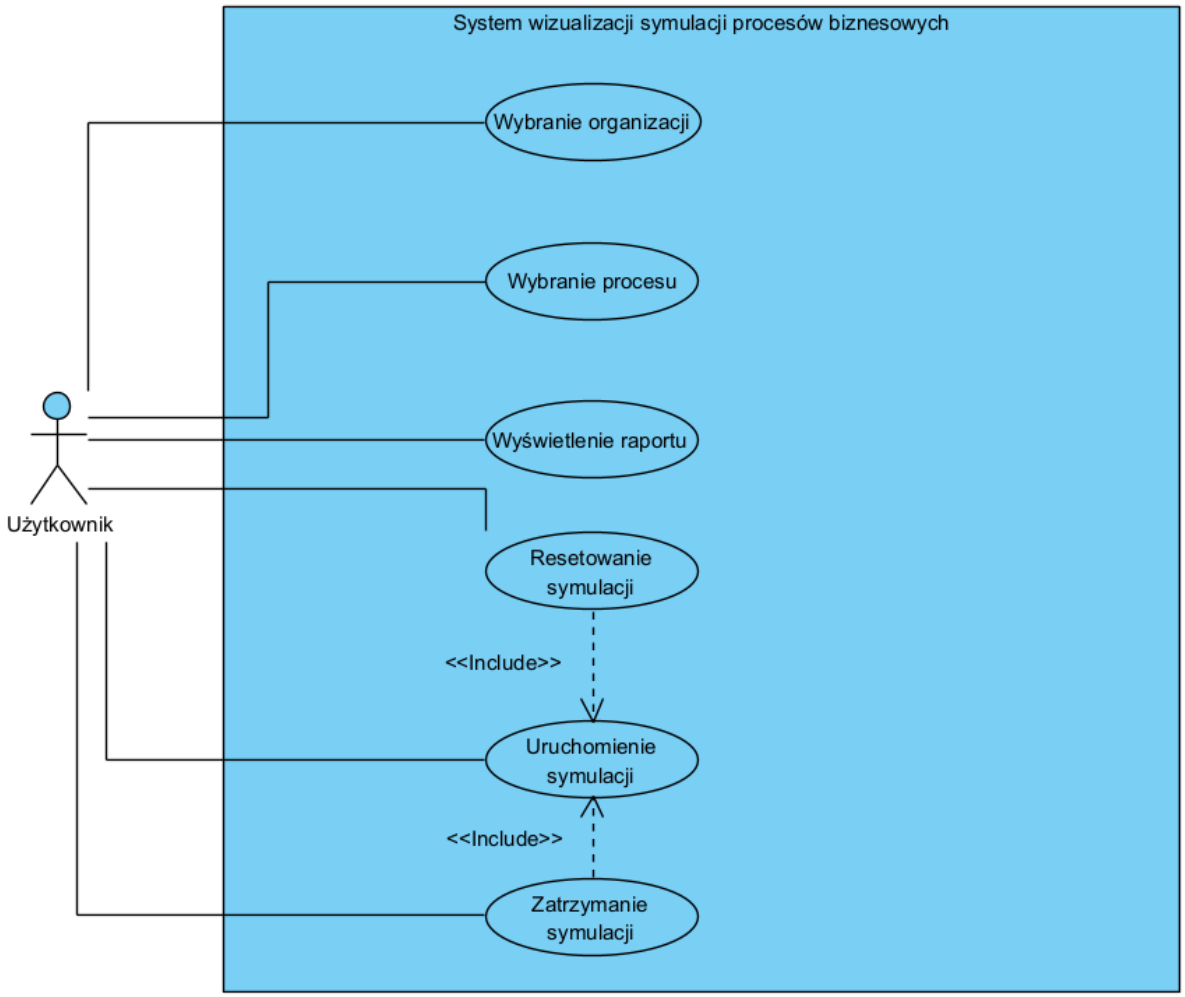
Diagram przypadków użycia jest graficznym przedstawieniem relacji między przypadkami użycia i aktorami. Relacje są pokazane jako linie, a aktorzy są przedstawiani jako ikony odwzorowujące człowieka. Przypadkami użycia są funkcjonalności jakie posiada system. Diagram przypadków użycia został przedstawiony na rysunku 7.

Użytkownikowi wyświetla się lista dostępnych organizacji, z której może dokonać wybrou interesującej go opcji. Na liście zostają wyświetlone nzwy frim. Po zaznaczeniu organizacji na liście użytkownikowi wyświetlają się informacje o organizacji czyli jakie procesy są w firmie.

Po wyborze organizacji użytkownikowi pokazuje się lista procesów biznesowych, które można symulować. Lista procesów w firmie jest dynamicznie generowana w oparciu o wcześniejszy wybór organizacji. Po wyborze procesu użytkownikowi pokazuje się graficznie przedstawienie procesu. Po wyświetleniu procesu użytkownik posiada możliwość dalszych zmian pocesu, ponieważ symulowana jest cała organizacaja a nie tylko jeden jej proces. Dowolna zmiana między procesami umożliwia wygodny podgląd procesów organizacji podczas trwania symulacji.

Przypadek użycia uruchomienie symulacji umożliwia uruchomienie symulacji całej organizacji poprzez naciśnięcie przycisku. Po uruchomieniu symulacji użytkownik może skorzystać z dwóch funkcjonalności takiej jak zatrzymanie symulacji i resetowanie symulacji, opcje te są dostępne podczas trwania symulacji. Stop symulacji zatrzymuje trwającą symulację, umożliwia to analizę graficznego przedstawienia procesów w chwili naciśniecia przysiku zatrzymania. Resetowanie symulacji przywraca graficzne przedstawienie procesów do sytuacji sprzed uruchomienia symulacji.

Po zakończeniu symulacji nastąpuje wyświetlenie raportu symulacji. Raport zawiera szczegółowe informacje na temat procesów i czynności zawartych w procesach organizacji.



Rys. 7. Diagram przypadków użycia

Źródło: opracowanie własne

I.4. Wymagania pozafunkcjonalne

Wymagania pozafunkcjonalne odnoszą się do jakości i operacyjnych aspektów systemu. Są to parametry, które definiują, jak system powinien działać, a nie co powinien robić. Wpływają na różne aspekty systemu: wydajność, niezawodność, używalność, kompatybilność. Są one kluczowe dla zapewnienia, że system będzie nie tylko spełniał oczekiwane funkcje, ale również był przyjazny dla użytkownika.

Aspekt wydajności odnosi się do szybkiego i efektywnego przetwarzania zleconych czynności. System reaguje na integrację z użytkownikiem w czasie nieprzekraczającym dwóch sekund. Symulacja przeprowadzana jest w czasie rzeczywistym. Aplikacja jest zdolna do obsługi co najmniej tysiąca użytkowników bez znaczącego spowolnienia działania aplikacji. W systemie istnieje możliwość dodawania nowych funkcjonalności, czyli rozbudowy aplikacji. Co pozwala na dopasowywanie się aplikacji do zmieniających się potrzeb użytkowników.

Niezawodność odnosi się do prawidłowego działania systemu, czyli jego odporności na błędny i awarie. Oznacza to, że aplikacja powinna być zdolna do pracy w różnych warunkach. System powinien skutecznie zarządzać błędami. Aplikacja powinna być dostępna przez 93% czasu.

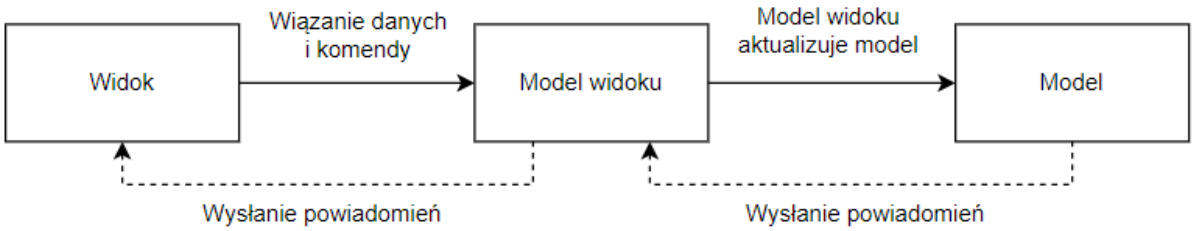
Aspekt używalności dotyczy łatwości i intuicyjności korzystania z systemu przez użytkownika. Interfejs użytkownika powinien być łatwy w obsłudze, intuicyjny i zgodny ze standardami UX (z ang. User Experience), czyli doświadczenie użytkownika. Co umożliwia nowym użytkownikom korzystanie z systemu bez konieczności odbywania długich szkoleń.

Kompatybilność to możliwość współpracy z innymi systemami. Aplikacja jest dostępna na przeglądarkach internetowych: Chrome od wersji 120, Firefox od wersji 121 i Edge od wersji 120. System jest responsywny, czyli dopasowuje się do różnych rozmiarów ekranów.

Rozdział II. Struktura aplikacji i integracji

II.1. Architektura aplikacji

MVVM (z ang. Model-View-ViewModel) – to wzorzec projektowy, który rozdziela logikę biznesową aplikacji (model) od interfejsu użytkownika (widoku), za pomocą warstwy pośredniczącej zwanej modelem widoku. Rysunek architektury znajduje się na rysunku 8.



Rys. 8. Architektura MVVM

Źródło: opracowanie własne

Warstwa modelu bezpośrednio zarządza danymi, logiką i regułami aplikacji. Model może być reprezentowany przez klasy TypeScript definiujące struktury danych, które są używane w całej aplikacji. Serwisy wykonują operacje na danych, takie jak zapytania HTTP do API. Serwisy mogą także zawierać logikę transformacji i przetwarzania danych przychodzących z serwera zanim zostaną one przekazane do modelu danych.

Warstwa widoku w architekturze MVVM odpowiada za prezentację interfejsu użytkownika. Jej głównym zadaniem jest wyświetlanie danych dostarczonych przez widok modelu w sposób, który jest zrozumiały i interaktywny dla użytkownika. Widoki są tworzone przy użyciu plików HTML i arkuszy stylów CSS. Definiują one strukturę i wygląd interfejsu użytkownika, włączając w to elementy takie jak przyciski, pola tekstowe, listy wyboru i inne komponenty. Warstwa widoku obsługuje wszelkiego rodzaju interakcje użytkownika, takie jak kliknięcia przycisków, wprowadzanie danych czy wybór opcji z listy rozwijanej. Wszystkie te akcje są przekazywane do modelu widoku, który następnie są tam przetwarzane.

Warstwa modelu widoku działa jako pośrednik między modelem a widokiem. Jest odpowiedzialna za przetwarzanie danych z modelu w sposób, który może być łatwo wyświetlony w widoku. Zawiera logikę prezentacji, która przekształca surowe informacje z modelu na format przyjazny dla użytkownika. Może to obejmować formatowanie dat, liczb, manipulowanie ciągami znaków i inne. Warstwa ta reaguje na akcje użytkownika zgłaszane przez widok, wykonuje odpowiednie operacje na danych i aktualizuje widok. Może także reagować na zmiany w danych, automatycznie aktualizując widok. Warstwa modelu widoku nie powinna zawierać bezpośrednich odniesień do elementów interfejsu użytkownika, aby zachować czystą separację między warstwą prezentacji a logiką biznesową.

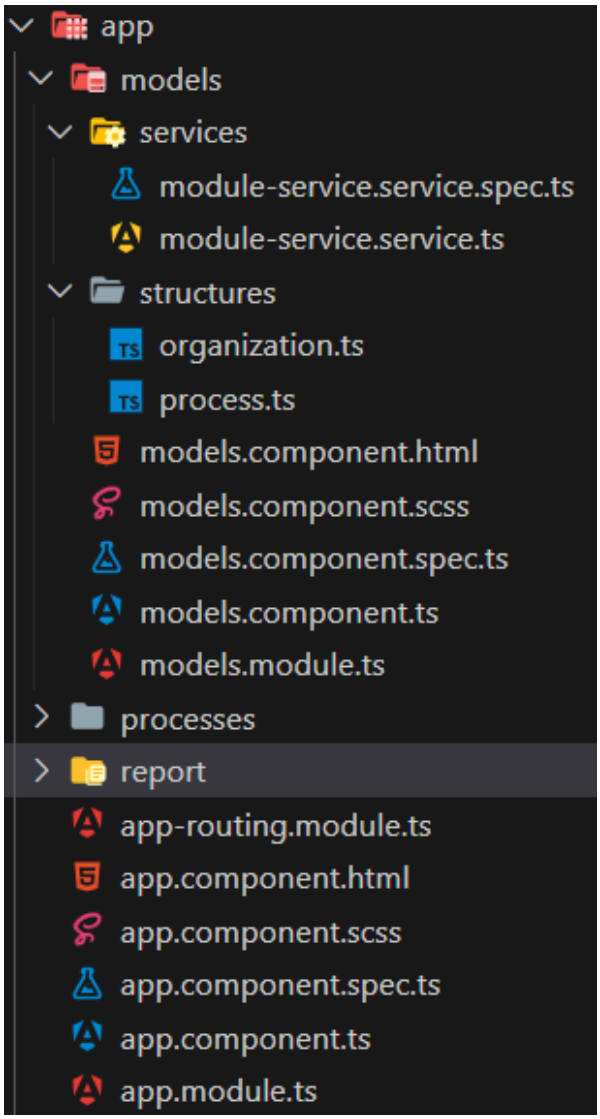
Struktura aplikacji została przedstawiona na rysunku 9. Katalogi: models, processes i report oddzielają pliki związane z konkretnymi interfejsami użytkownika:

- models – pliki związane z ekranem wyboru organizacji,
- processes – pliki wiązane z ekranem symulacji,
- report – pliki związane z ekranem raportu.

W katalogu models znajdują się dwa katalogi: services i structures, odpowiadają one warstwie modelu. W folderze services znajduje się serwis do komunikacji z API. W folderze structures znajdują się pliki TypseScript, które definiują struktury danych:

- organization.ts – model danych organizacji,
- proces.ts – model danych procesu.

Przetwarzanie danych uzyskanych z warstwy modelu odbywa się w pliku models.component.ts. Plik ten odpowiada warstwie modelu widoku. Znajdują się tu jeszcze reakcje na działania użytkownika, takie jak naciśniecie przycisku. Przetworzone dane wyświetlane są poprzez plik models.component.html, zawiera on zdefiniowaną strukturę i elementy interfejsu wyboru organizacji, takie jak przyciski i lista jednokrotnego wyboru. Plik models.component.scss zawiera informacje jak elementy interfejsu powinny wyglądać, takie jak kolor przycisków. Pliki models.component.html i models.component.scss odpowiadają warstwie widoku. Katalogi odpowiedzialne za pozostałe ekrany zostały zaprojekowane w podobny sposób.



Rys. 9. Struktura aplikacji

Źródło: opracowanie własne

Aplikacja została napisana przy użyciu Angulara, jest to narzędzie do tworzenia interaktywnych, responsywnych interfejsów użytkownika. Wykorzystanie TypeScript w ramach Angulara zapewnia silne wsparcie dla typowania, co przekłada się na lepszą organizację kodu i łatwiejsze zarządzanie projektem. [6]

ngx-graph to biblioteka Angulara umożliwiająca tworzenia złożonych wizualizacji grafów. W aplikacji biblioteka ta jest wykorzystywana do dynamicznego tworzenia i manipulowania reprezentacją graficzną procesów. Umożliwia ona tworzenie interaktywnych grafów, które mogą być łatwo modyfikowane w czasie rzeczywistym, co jest kluczowe przy symulacji przepływu procesów. [5]

Biblioteka d3-shape jest wykorzystywana do rysowania połączeń między elementami grafu. W kontekście symulacji procesów biznesowych, pozwala na efektywne przedstawienie przepływu między czynnościami i bramkami logicznymi. Pomaga to w analizie procesów biznesowych. [8]

Angular Material Icons to biblioteka ikon, która zapewnia szeroki wybór graficznych reprezentacji do użytku w interfejsach użytkownika. W aplikacji do symulacji procesów biznesowych, ikony te są używane do wizualnego przedstawienia różnych rodzajów zadań i bramek logicznych w procesie. Ich zastosowanie nie tylko podnosi estetykę aplikacji, ale także ułatwia zrozumienie i interakcję z symulacją. [9]

RxJS to biblioteka programowania reaktywnego, która pozwala na zarządzanie strumieniami danych i asynchronicznymi zdarzeniami. Umożliwia na implementację typu Observable, która pozwala traktować zdarzenia asynchronicznie i umożliwia użycie operatorów do transformowania danych. [10]

II.2. Model danych

Model danych aplikacji został przedstawiony na rysunku 10. Struktura danych na temat symulowanej organizacji została przedstawiona w tabeli Organization. Zawarte w niej nazwy pól oznaczają:

- id – numer indeksu organizacji,
- name – nazwa organizacji,
- sub_organization – nazwa pod organizacji.

Organizacje posiadają procesy, struktura danych procesów została przedstawiona w tabeli Processes, nazwy jej atrybutów oznaczają:

- id – numer indeksu procesu,
- name – nazwa procesu,
- simulation – numer indeksu symulacji,
- organization – numer indeksu organizacji,
- generator – numer indeksu generatora.

W Procesie zawarta jest informacja o numerze indeksu symulacji. Model danych symulacji został przedstawiony w tabeli Simulation. Nazwy atrybutów w modelu oznaczają:

- id – numer indeksu symulacji,
- name – nazwa symulacji,
- initial_time – czas rozpoczęcia symulacji,
- frequency – częstotliwość symulacji,
- end_time – czas zakończenia symulacji.

Podczas symulacji generowane są hipotetyczne instancje dla każdego procesu organizacji. Nazwy pól w modelu Generator oznaczają:

- id – numer indeksu generatora,
- name – nazwa generatora,
- max_transaction – maksymalna liczba wygenerowanych hipotetycznych jednostek.,
- stop_time – czas zatrzymania generatora,

- delay_time – czas opóźnienia startu generatora,
- time – początek zakresu czasu losowania,
- time_range_end – koniec zakresu czasu losowania,
- time_type – rozkład prawdopodobieństwa generowania instancji procesu,
- monitor_generated – ilość wygenerowanych jednostek,
- event – numer indeksu czynności.

Organizacja posiada zasoby, zostały one przedstawione w tabeli Resource. Zawarte w niej nazwy pól oznaczają:

- id – numer indeksu zasoby,
- name – nazwa zasobu,
- type – typ zasobu,
- quantity – ilość zasobów,
- cost_of_use – koszt zasobu,
- hourly_salary – godzinne wynagrodzenie,
- organization – numer indeksu organizacji.

Czynności w procesie są realizowane przez zasoby, model danych zwierająca informacje na temat czynności został przedstawiony w tabeli Event. Nazwy jej pól oznaczają:

- id – numer indeksu czynności,
- name – nazwa czynności,
- type – zadanie takie jak: nieokreślone, użytkownika, manualne, usługowe,
- num_employees – liczba zasobów potrzebnych do realizacji zadania,
- time_range_end – czas zakończenia możliwego wykonywania zadania,
- time_type – rozkład prawdopodobieństwa pojawiania się instancji procesu,
- monitor_pending – liczba oczekujących procesów,
- monitor_execute – liczba realizowanych procesów,
- monitor_realized – liczba zrealizowanych procesów,
- output – numer indeksu następnego elementu,
- resource – numer indeksu zasobu.

Czynności wskazują na następny element w procesie, może być nim bramka Or. Jej struktura została przedstawiona w tabeli OrGateway, nazwy jej atrybutów oznaczają:

- id – numer indeksu bramki OR,
- name – nazwa bramki,
- precentages – numer indeksu następnego elementu i procentowa szansa na realizacje elementu,
- resource – numer indeksu zasobu.

Bramka OR zawiera informacje o następnych elementach w procesie. Może nim być bramka XOR. Zawarte nazwy zmiennych w modelu XorGateway oznaczają:

- id – numer indeksu bramki XOR,

- name – nazwa bramki,
- precentages – numer indeksu następnego elementu i procentowa szansa na realizacje elementu,
- resource – numer indeksu zasobu.

Następnym elementem wskazywanym przez czynność może być bramka AND. Struktura danych zawierająca informacje na jej temat została przedstawiona w tabeli ParallelGateway. Nazwy pól oznaczają:

- id – numer indeksu bramki AND,
- name – nazwa bramki,
- Id_list – lista numerów indeksów następnych elementów,
- resource – numer indeksu zasobu.



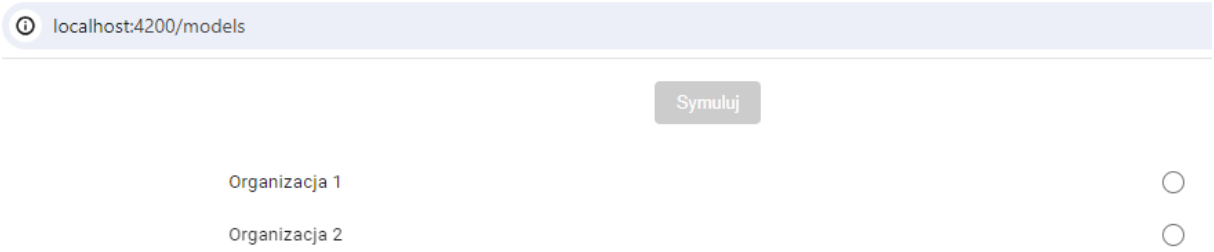
Rys. 10. Model danych aplikacji

Źródło: opracowanie własne

Rozdział III. Opis implementacji

III.1. Ekran wyporu organizacji

Interfejs wyboru organizacji został przedstawiony na rysunku 11. Umożliwia użytkownikom interaktywny wybór organizacji, dla której chcą przeprowadzić symulację procesów biznesowych. Ekran wyboru organizacji aplikacji służy jako punkt wejściowy dla użytkownika do dalszych etapów symulacji.



Rys. 11. Interfejs wyboru organizacji

Źródło: opracowanie własne

Na ekranie znajduje się lista wyboru organizacji. Fragment implementacji listy znajduje się w kodzie 1. „[multiple] = false" oznacza, że możliwe jest zaznaczenie tylko jednego elementu z listy. W parametrze „value" jest przechowywany unikany identyfikator wiersza, czyli indeks organizacji. „*ngFor" służy do iteracji po kolejnych nazwach organizacji i ich wyświetlaniu.

Kod. 1. Lista jednokrotnego wyboru organizacji

```
1   <mat-selection-list #org [multiple]="false" class="list-
    container">
2       <mat-list-option *ngFor="let organization of
    organizationList" [value]="organization.id">
3         {{organization.name}}
4       </mat-list-option>
5     </mat-selection-list>
```

Źródło: opracowanie własne

Po wybraniu organizacji, na liście wyświetlane są dostępne procesy wybranej przez użytkownika firmie. Warunek wyświetlania listy przedstawiony został w kodzie 2. Lista procesów wyświetli się, gdy została wybrana organizacja z listy i lista procesów nie jest pusta.

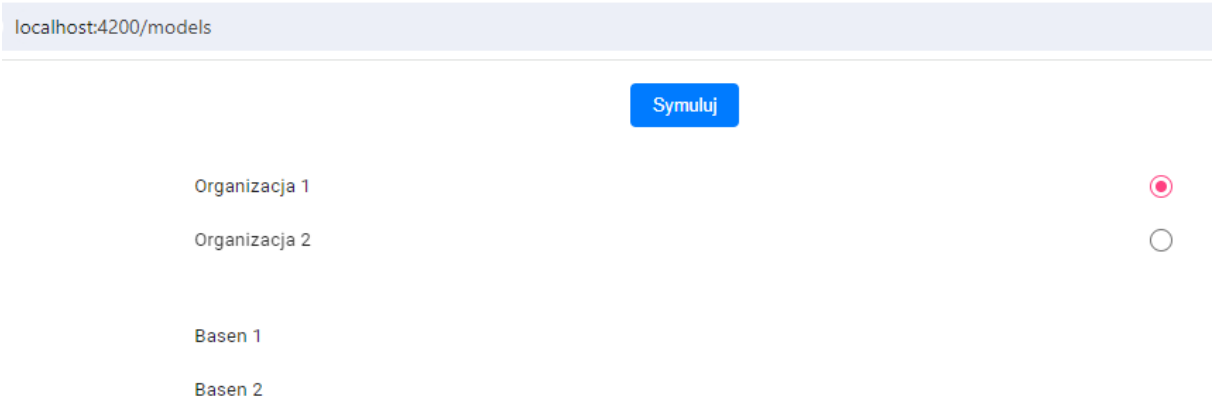
Kod. 2. Warunek wyświetlania listy procesów organizacji

```
1     <mat-list *ngIf="org.selectedOptions.hasValue() &&
    processesList" class="list-container">
```

Źródło: opracowanie własne

Wygląd ekranu po wyborze organizacji przedstawiony został na rysunku 12. Przycisk symulacji pozwala na przejście do ekranu symulacji dla zaznaczonej

organizacji. Przycisk ten staje się aktywny dopiero po wybraniu organizacji z listy. Po kliknięciu aktywnego przycisku, użytkownik jest przekierowywany do ekranu symulacji. Wraz ze zmianą ścieżki znajduje się w niej informacja o indeksie organizacji. Fragment kodu przedstawiający przycisk „Symuluj" znajduje się w kodzie 3.



Rys. 12. Interfejs użytkownika po wyborze organizacji

Źródło: opracowanie własne

Kod. 3. Przycisk „Symuluj"

```
<button class="btn btn-primary custom-button"
[disabled]="!org.selectedOptions.hasValue()"
[routerLink]="['/processes',
      org.selectedOptions.hasValue() ?
      org.selectedOptions.selected[0].value : '']">
  Symuluj
</button>
```

Źródło: opracowanie własne

Serwis ModuleServiceService jest odpowiedzialny za komunikację z API. Wykorzystuje on HttpClient do wykonywania zapytań HTTP i pobierania danych organizacji oraz procesów. Metody get() i getProcesses() służą do pobierania informacji o organizacjach i procesach, metody zostały przedstawione w kodzie 4.

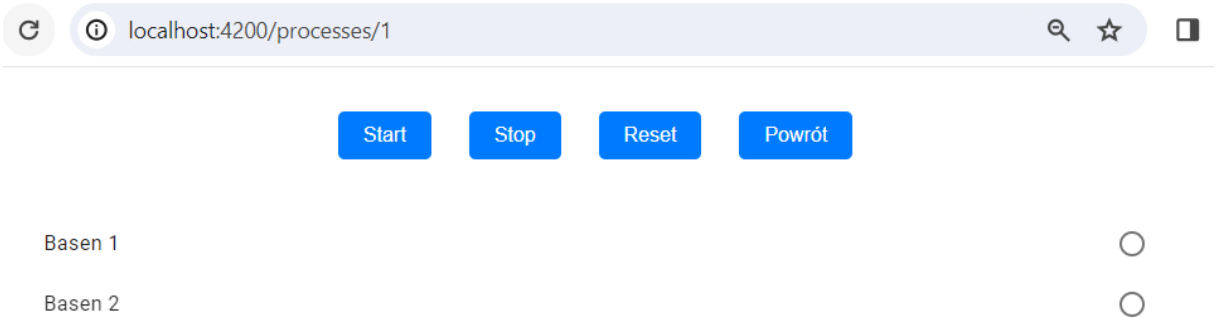
Kod. 4. Metody do pobierania danych o organizacjach i procesach

```
Get(): Observable<any> {
      Return this.http.get<OrganizationList[]>
      ('http://localhost:8000/resources/organizations/');
}
getProcesses(): Observable<any> {
      Return this.http.get<OrganizationList[]>
      ('http://localhost:8000/processes/');
}
```

Źródło: opracowanie własne

III.2. Ekran symulacji

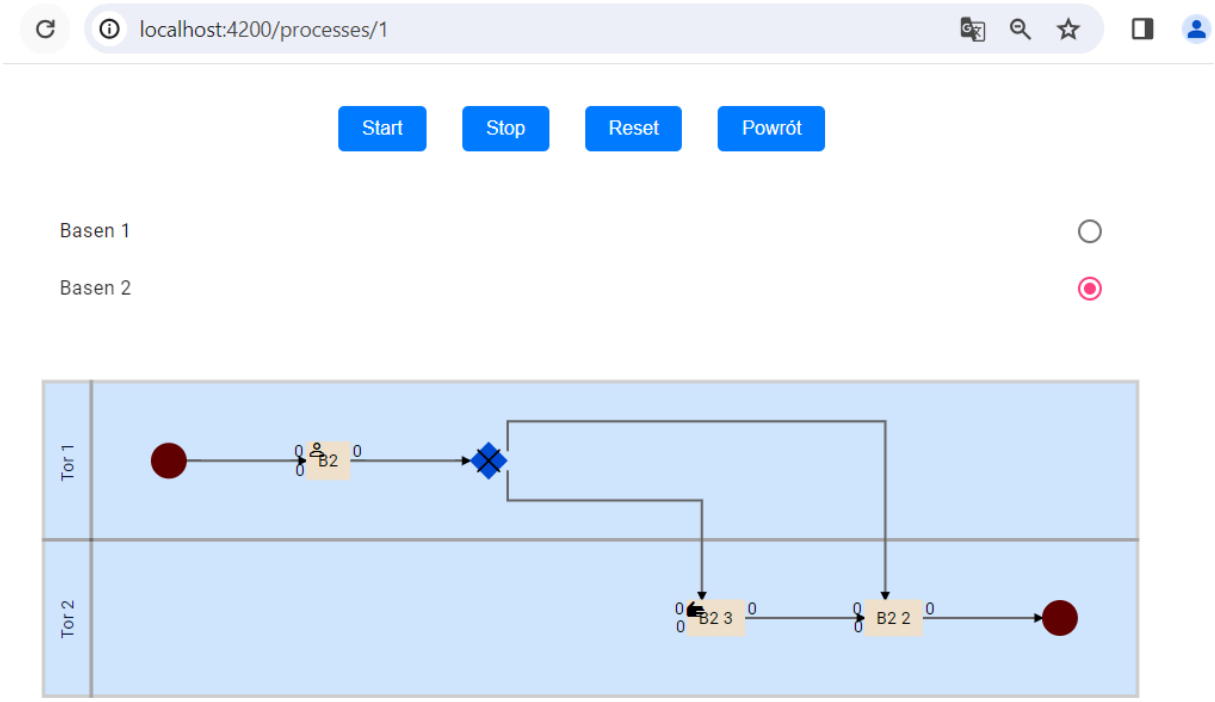
Ekran symulacji został przedstawiony na rysunku 13. Na interfejsie znajdują się przyciski i lista procesów wybranej wcześniej organizacji.



Rys. 13. Ekran symulacji przed wyborem procesu

Źródło: opracowanie własne

Po wybraniu procesu z listy wyświetla się model procesu biznesowego, przedstawiony na rysunku 14. Użytkownik ma możliwość zmiany wyświetlanego procesu. Przyciski: „Start”, „Stop” i „Reset” umożliwiają użytkownikowi sterowanie przebiegiem symulacji. Przycisk „Start” uruchamia proces symulacji, co skutkuje rozpoczęciem animacji i aktualizacją danych. Przycisk „Stop” zatrzymuje symulację, użytkownik może przeanalizować stan procesów. Przycisk „Reset” przywraca stan początkowy symulacji, czyści wszelkie dotychczasowe informacje i umożliwia ponowne uruchomienie symulacji od początku. Przycisk „Powrót” przywraca wyświetlanie poprzedniego widoku, czyli ekranu wyboru organizacji.



Rys. 14. Ekran symulacji po wybraniu procesu

Źródło: opracowanie własne

Proces jest wyświetlany przy pomocy biblioteki ngx-graph do wizualizacji grafów. Struktura programu użycia biblioteki znajduje się w kodzie 5.

Kod. 5. Struktura programu ngx-graph

```
<ngx-graph
  [links]="krawędzie"
  [nodes]="węzły"
  [clusters]="klastry"
  [layout]="ułożenie"
  [curve]="łuk" >
     <ng-template>
           Kod SVG
     </ng-template>
</ngx-graph>
```

Źródło: opracowanie własne

Węzły to lista elementów typu Node, zawiera informacje o wierzchołkach grafów takie jak: unikalny indeks, nazwa, wymiary wierzchołka, pozycja, kolor oraz dodatkowe informacje. W programie wierzchołkami są czynności i bramki. Informacje o tym czy wierzchołek reprezentuje bramkę bądź czynność jak również pozostałe informacje o nich, są zapisywane w dodatkowych informacjach o wierzchołku.

Krawędzie to lista elementów typu Edge, która zawiera informacje na temat łączenia wierzchołków w grafie. Każda krawędź zawiera informacje o indeksie wierzchołka grafu, z którego wychodzi początek linii i indeks wierzchołka, do którego zmierza.

Klastry to lista elementów typu ClusterNode. W klastrach węzły są grupowane w większe jednostki, co umożliwia lepszą organizacje i uporządkowanie grafu. W przypadku aplikacji klastrami są tory pływackie. Klaster zawiera informacje takie jak unikalny indeks, nazwa i tablica zawierająca indeksy wierzchołków grafu, które są przypisane do klastra.

Ułożenia definiują sposób w jaki węzły są rozmieszczone na grafie. Biblioteka ngx-graph udostępnia szablony ułożenia grafu. Istnieje również możliwość tworzenia niestandardowych ułożeń, przez stworzenie własnej klasy, która implementuje interfejs Layout.

Łuk jest sposobem przedstawienia połączenia między węzłami. Kształt i styl mogą być modyfikowane w celu osiągnięcia pożądanych efektów wizualnych, oraz lepszego przedstawienia relacji między elementami grafu. Można skorzystać z krzywych dostarczanych przez bibliotekę d3-shape, istnieje również możliwość tworzenia niestandardowych ułożeń krzywych.

Szablony SVG umożliwiają definiowanie niestandardowych wyglądów węzłów i krawędzi. SVG (z ang. Scalable Vector Graphics) – format grafiki wektorowej, który jest używany do opisywania obrazów, które są w dwóch wymiarach. Szablony mogą zawierać dynamiczne informacje i reagować na interakcje użytkownika. Umożliwiają przekształcenie grafu przez nadanie odpowiednich kształtów i kolorów elementom grafu.

Klasa MyLayout jest niestandardowym ułożeniem grafu, która jest modyfikacją ułożenia dostarczonego przez bibliotekę ngx-graph. Modyfikuje ona wygląd grafu tak, aby wierzchołki grafu i klastry były odpowiednio ułożone. Domyślne ustawienia grafu znajdują się w kodzie 6. Poszczególne elementy ustawień grafu oznaczają:

- orientation – określa kierunek, w jakim graf zostanie ułożony. Są cztery możliwości ułożenia: od lewej do prawej, od prawej do lewej, od góry do dołu i od dołu do góry. W przypadku aplikacji jest to od lewej do prawej, ponieważ układ torów pływackich jest poziomy i początek znajduje się z lewej strony.
- marginX i marginY - oznaczają marginesy na zewnątrz grafu. marginX odpowiada za marginesy z lewej i prawej stronie grafu, a marginY za marginesy z górnej i dolnej strony.
- edgePadding – określa odstęp między klastrami grafu.
- rankPadding – oznacza odstęp poziomy między kolejnymi wierzchołkami w grafie.
- nodePadding – określa odstęp między węzłami w pionie.
- multigraph – wartość „true” czyli prawda wskazuje, że pomiędzy parą wierzchołków w grafie może być wiele krawędzi.
- compound – określa tworzenie grafów złożonych, czyli umożliwia tworzenie klastrów. Ustawienie „true” oznacza, że opcja jest włączona.

- ranker – jest to metoda używana do ustalania poziomów węzłów. „longesth-path” oznacza, że wierzchołki grafu są ustalane na podstawie najdłuższej ścieżki od poprzedniego wierzchołka.

Kod. 6. Domyślne ustawienia grafu

```
defaultSettings: DagreSettings = {
        orientation: Orientation.LEFT_TO_RIGHT,
        marginX: 40,
        marginY: 10,
        edgePadding: 0,
        rankPadding: 100,
        nodePadding: 100,
        multigraph: true,
        compound: true,
        ranker: 'longest-path',
      };
```

Źródło: opracowanie własne

Metoda run w klasie myLayout jest odpowiedzialna za ułożenie elementów w grafie. Najpierw obliczane jest położenie dla wszystkich węzłów i krawędzi w grafie zgodnie z domyślnymi ustawieniami. Następnie funkcja dagereToOutput aktualizuje wierzchołki, a dagereToOutput2 klastry. Funkcja dagereToOutput została przedstawiona w kodzie 7. W funkcji jest zapisywany identyfikator klastra z pierwszą czynnością. Następnie sprawdzane jest, czy wielkość wierzchołka, który jest bramką jest odpowiednia duża, jeżeli nie to modyfikuje wielkość. Zwracane są zmodyfikowane węzły.

Kod. 7. Funkcja dagereToOutput

```
        const dagreToOutput = (node:Node) => {
          const dagreNode = this.dagreGraph._nodes[node.id];
        if(Object.keys(this.dagreGraph._preds[node.id]).length
=== 0){
          this.clusterID = this.dagreGraph._parent[node.id]
        }
        if(dagreNode.data.shape === 'diamond' &&
dagreNode.dimension.height< 40 &&
dagreNode.dimension.width<40){
          dagreNode.dimension.height= 1.5 *
dagreNode.dimension.height;
          dagreNode.dimension.width= 1.5 *
dagreNode.dimension.width;
        }
          return {
            ...node,
            position: {
              x: dagreNode.x,
              y: dagreNode.y
```

```
15              },
16              dimension: {
17                width: dagreNode.width,
18                height: dagreNode.height
19              }
20            };
21          };
```

Źródło: opracowanie własne

Funkcja dagereToOutput2 została pokazana w kodzie 8, zwraca zmodyfikowane klastry. Korzysta z identyfikatora klastra, w którym znajduje się pierwsza czynność, ponieważ klastry powinny być tej samej długości a klaster ten po obliczeniu położeń elementów grafu jest najdłuższy.

Kod. 8. Funkcja dagereToOutput2

```
1   const dagreToOutput2 = (node:Node) => {
2             const dagreNode = this.dagreGraph._nodes[node.id];
3             const dagreNode2 =
    this.dagreGraph._nodes[this.clusterID];
4             return {
5               ...node,
6               position: {
7                 x: dagreNode2.x,
8                 y: dagreNode.y
9               },
10              dimension: {
11                width: dagreNode2.width,
13                height: dagreNode.height
14              }
15            };
16          };
```

Źródło: opracowanie własne

myCurve to specjalnie zdefiniowana klasa, która rysuje niestandardowe krawędzie pomiędzy węzłami grafu. Klasy myCurve nie ma w dostarczonych stylach przez bibliotekę d3-shape, ale jest modyfikacją linii klasy step. Kluczowym elementem do rysowania krzywej jest funkcja point przedstawiona została w kodzie 9. Przyjmuje ona dwa parametry: x jest to liczba określająca współrzędną poziomą punktu i y współrzędną pionową. Funkcja wykorzystuje instrukcję switch, aby obsługiwać różne scenariusze rysowania punktów. Każdy przypadek odpowiada różnym etapom rysowania ścieżki: początek, kontynuacja i zakończenie. Jeśli funkcja point jest wywoływana po raz pierwszy dla ścieżki, ustawia początkowy punkt ścieżki. Przy drugim wywołaniu, łączy początkowy punkt z nowym punktem, tworząc linię. Dla każdego kolejnego punktu, funkcja łączy ostatni punkt z nowym punktem, kontynuując krzywą.

Kod. 9. Funkcja point

```
1   point(x: number, y: number) {
2         x = +x, y = +y;
3         switch (this._point) {
4           case 0: this._point = 1;
5             this._line ? this._context.lineTo(x, y) :
    this._context.moveTo(x, y);
6             break;
7           case 1: this._point = 2;
8             this._context.lineTo(this._x, y);
9             this._context.lineTo(x, y);
10            break;
11          default: {
13            this._context.lineTo(x, this._y);
14            this._context.lineTo(x, y);
15            break;
16          }
17        }
18        this._x = x, this._y = y;
```

Źródło: opracowanie własne

Po przejściu do ekranu symulacji, pobierany jest parametr processesid z aktualnej ścieżki URL. Uzyskanie parametru ze ścieżki URL przedstawiono w kodzie 10. Jest to identyfikator organizacji który został dodany do ścieżki w poprzednim ekranie wyboru organizacji. Parametr jest konwertowany na liczbę. Następnie za pośrednictwem serwisu ProcessesServices pobierane są informacje symulacji. Pobranie procesów organizacji zostało przedstawione w kodzie 11, gdzie następuje wywołanie usługi HTTP i filtrowanie otrzymanych danych tak aby przekazać tylko te procesy, które należą do wybranej organizacji.

Kod. 10. Pobranie identyfikatora organizacji

```
1   const idString =
    this.route.snapshot.paramMap.get('processesid');
```

Źródło: opracowanie własne

Kod. 11. Pobranie informacji o procesach

```
1   getProcesses(id: number): Observable<any> {
2       return
    this.http.get<ProcessList[]>('http://localhost:8000/processes/')
3         .pipe(
4           map((processes: ProcessList[]) =>
    processes.filter(process => process.organization === id))
5         );
6     }
```

Źródło: opracowanie własne

Po pobraniu wszystkich informacji są one przypisywane do zmiennych. Następnie do zmiennej pollList, która zawiera strukturę basenu. Dodawane są czynności, które są generatorami. Dla każdego procesu ze zmiennej pollList wywołana jest funkcja step, która przetwarza elementy basenu. Funkcja dodaje czynności do listy procesów, wiązania i przypisuje do odpowiednich klastrów. Po czym wywołuje kolejny raz funkcję step, dla następnego elementu.

Po naciśnięciu przycisku „Start” w aplikacji wywoływana jest metoda onStart. Wywołuje ona funkcję getRun z serwisu ProcessesService, z argumentem idSimulation, czyli identyfikatorem organizacji. Funkcja ta wysyła zapytanie HTTP w celu rozpoczęcia symulacji. W odpowiedzi dostaje identyfikator symulacji. Następuje inicjalizacja i wywołanie asynchronicznej funkcji symulacji, która co sekundę pobiera informacje o czynnościach i aktualizuje wszystkie czynności w każdym procesie. Fragment aktualizujący wszystkie czynności w procesie zawarty został w kodzie 12. W data są przechowywane dodatkowe informacje: monitor_pending – liczba czynności oczekujących na realizację, monitor_execute – liczba czynności realizowanych, monitor_realized – liczba czynności zrealizowanych.

Kod. 12. Aktualizacja liczników czynności

```
this.pollList.forEach(element => {
            element.poll.nodes.forEach( eve => {
              if( eve.data.shape === ""){
                let event = this.eventList.find(value =>
value.id.toString() === eve.id);
                if( event !== null ){
                  eve.data.monitor_pending=
event!.monitor_pending;
                  eve.data.monitor_execute=
event!.monitor_execute;
                  eve.data.monitor_realized=
event!.monitor_realized;
                }
              }
            })
```

Źródło: opracowanie własne

Po aktualizacji następuje sprawdzenie czy symulacja się zakończyła, jest ono pokazane w kodzie 13. Jeżeli symulacja się zakończyła to pętla symulacji odpytującej się o stan czynności co sekundę zostaje przerwana, przez zmianę wartości zmiennej isRunnging i użytkownik zostaje przekierowany do ekranu raportu. Podczas zmiany adresu URL dodawany jest identyfikator symulacji.

Kod. 13. Sprawdzenie statusu symulacji

```
this.processesService.getRunningSimulation(this.idRunningSimulat
ion!).subscribe(async data => {
            if( data === "SUCCEED"){
```

```
3                this.isRunning = false;
4
  this.router.navigateByUrl(`/report/${this.idRunningSimulation}`)
  ;
5              }
6            });
```

Źródło: opracowanie własne

Przycisk „Reset” umożliwia przywrócenie stanu symulacji do początkowego ustawienia. Po jego naciśnięciu uruchamiana jest funkcja onReset, która przerywa pętlę symulacji odpytującej się o stan czynności co sekundę, przez zmianę wartości zmiennej isRunnging. Wysyłane jest zapytanie HTTP przez serwis ProcessesService, które ma zresetować symulację. Następnie liczniki wszystkich czynności są zerowane. Fragment z zerowaniem liczników czynności zawarty jest w kodzie 14.

Kod. 14. Przywrócenie liczników czynności do stanu początkowego

```
1   this.pollList.forEach(element => {
2       element.poll.nodes.forEach( eve => {
3         if( eve.data.shape === ""){
4             eve.data.monitor_pending= 0;
5             eve.data.monitor_execute= 0;
6             eve.data.monitor_realized= 0;
7         }
8       })
9     });
```

Źródło: opracowanie własne

Naciśnięcie przycisku „Stop” wywołuje funkcje onStop. Pętla symulacji zostaje przerwana i zostaje wysłane zapytanie HTTP, które informuje o zatrzymaniu symulacji. Przycisk „Powrót” pozwala użytkownikowi na powrót do poprzedniego ekranu wyboru organizacji. Kod przycisku przedstawiony jest w kodzie 15.

Kod. 15. Przycisk „Powrót”

```
1   <button routerLink="/models" class="btn btn-primary custom-
    button">Powrót</button>
```

Źródło: opracowanie własne

Szablony SVG umożliwiają wizualną modyfikacje grafu. Wierzchołki grafu w dodatkowych elementach (data) zawierają informacje o kształcie(data.shape). „circle” – oznacza, że wierzchołkiem grafu jest zdarzenie startu bądź zakończenia. Informacja „diamond” mówi, że wierzchołkiem jest bramka, a brak informacji oznacza. że to czynność. Na podstawie kształtu w dodatkowych informacjach (data.shape) następuje nadanie kształtu wierzchołkowi. Fragment szablonu znajdującego się w kodzie 16 modyfikuje wygląd wierzchołków, które są zdarzeniami startu i zakończenia. Okrąg zostanie narysowany, jeżeli w informacjach dodatkowych o kształcie będzie wartość

„circle”. Atrybuty „cx” i „cy” określają współrzędne koła, „r” to promień, a „fill” to kolor wypełnienia koła.

Kod. 16. Modyfikacja wierzchołka grafu, będącego zdarzeniem startu lub zakończenia

```
<ng-template #nodeTemplate let-node>
    <svg:g class="node">
      <ng-container [ngSwitch]="node.data.shape">
        <svg:circle *ngSwitchCase="'circle'"
        [attr.cx]="node.dimension.width / 2"
        [attr.cy]="node.dimension.height / 2"
        [attr.r]="node.dimension.height/ 2"
        [attr.fill]="'#600000'"
      />
```

Źródło: opracowanie własne

Czynności są reprezentowane przez prostokąty, które są rysowane wtedy, gdy informacja o kształcie jest pusta, czyli nie zawierają żadnych znaków. Natomiast bramki są przedstawiane za pomocą znaku diamentu, które są rysowane, gdy informacja o kształcie w danych dodatkowych to „diamond”. Do tworzenia prostokątów wykorzystywana jest instrukcja rect, natomiast bramki są generowane przy użyciu instrukcji poligon.

Czynności mogą być różnego typu, informacje o tym jakiego typu jest zawarta w typie w informacjach dodatkowych wierzchołka (node.data.task). Informacja „USER” to zadanie użytkownika, „MANUAL” oznacza zadanie manualne, a „service” to zadanie usługowe. W zależności od tego jakie to zadanie w czynności zostaje wyświetlona odpowiednia ikona ją reprezentująca. Fragment kodu, który odpowiada za wyświetlenie ikony dla zadania użytkownika pokazany został w kodzie 17. Ikony dla pozostałych zadań pokazywane są w podobny sposób.

Kod. 17. Modyfikacja wyglądu zadania użytkownika

```
<svg:foreignObject
        *ngIf="node.data.task==='USER'"
        [attr.width]="node.dimension.width/2"
        [attr.height]="node.dimension.height/2"
        [attr.y]="'-2px'"
      >
        <div xmlns="http://www.w3.org/1999/xhtml" style="width:
100%; height: 100%;">
          <mat-icon style="font-size:
18px;">perm_identity</mat-icon>
        </div>
</svg:foreignObject>
```

Źródło: opracowanie własne

Gdy wierzchołkiem grafu jest bramka rysowany jest diament, ale bramki też są różnego rodzaju. Informacja o tym jakiego typu jest bramka znajduje się w informacjach

dodatkowych w bramce (node.data.gate). Informacje o typie bramki oznaczają: „or” – bramka typu OR, „and” – brama AND i „xor” – bramka typu XOR. Tak samo jak w przypadku czynności w zależności od typu bramki zostaje wyświetlona odpowiednia ikona.

W czynności wyświetlana jest jej nazwa, a po bokach informacje o przebiegu symulacji. W lewym górnym rogu znajduje się liczba procesów oczekujących na realizację, w prawym górnym rogu liczba realizowanych procesów, a w lewym dolnym rogu wyświetlana jest liczba procesów zrealizowanych. Nazwa i liczniki symulacji wyświetlane są, jeżeli wierzchołek jest czynnością. Przykład wyświetlenia nazwy znajduje się w kodzie 18.

Kod. 18. Wyświetlanie nazwy czynności

```
1   <svg:text *ngSwitchCase="''" alignment-baseline="central"
2   [attr.x]="10" [attr.y]="node.dimension.height / 2">
3         {{node.label}}
4    </svg:text>
```

Źródło: opracowanie własne

Podczas symulacji po bokach czynności aktualizowane są liczniki procesów oczekujących, realizowanych i zrealizowanych. Dodatkowo, jeżeli czynność jest realizowana to prostokąt zmienia kolor na zielony, a jeżeli oczekuje na zasób to na żółty. Przykład zmiany koloru czynności na zielony jest widoczny w kodzie 19. Kolor zmieni się na zielony, jeżeli liczba procesów realizowanych jest większa od zera. Podobnie w przypadku pokazania oczekiwania procesu na zasoby. Kolor zmieni się na jasno żółty, jeżeli liczba procesów realizowanych wynosi zero i liczba procesów oczekujących jest większa od zera.

Kod. 19. Zmiana koloru czynności na zielony

```
1   <svg:rect *ngIf="node.data.monitor_execute > 0 "
2             [attr.width]="node.dimension.width"
3             [attr.height]="node.dimension.height"
4             [attr.fill]="'#05F946'"
5         />
```

Źródło: opracowanie własne

Szablony umożliwiają również zmianę wyglądu linii między wierzchołkami grafu. Linia przepływu powinna być zakończona grotem. Fragment kodu definiujący znacznik używany jako strzałka na końcu linii znajduje się w kodzie 20. Atrybut „viewBox” definiuje obszar widzenia, w którym rysowany jest znacznik. „refX” i „refY” określają pozycję odniesienia znacznika w odniesieniu do linii. „markerWidth” i „markerHeight” oznaczają rozmiar znacznika. Wartość „auto” dla „orient” mówi, że orientacja znacznika ma się dopasowywać automatycznie do kierunku linii. Atrybut d = "M0,-5L10,0L0,5” określa ścieżkę rysunku. Seria instrukcji rysujących strzałkę:

- M0,-5 oznacza przeniesienia pióra do punktu (0,-5),

- L10,0 rysuje linię do punktu (10,0),
- L0,5 rysuje linię do punktu (0,5).

Końcowy efekt to trójkątna strzałka.

Kod. 20. Zdefiniowanie znaku grotu

```
<ng-template #defsTemplate>
    <svg:marker id="arrow" viewBox="0 -5 10 10" refX="8" refY="0" markerWidth="4" markerHeight="4" orient="auto">
      <svg:path d="M0,-5L10,0L0,5" class="arrow-head" />
    </svg:marker>
  </ng-template>
```

Źródło: opracowanie własne

Fragment kodu służący do generowania każdej krawędzi znajduje się w kodzie 30. class=" line" oznacza klasę CSS do stylizacji linii. „stroke-width" określa grubość linii. Marker-end wskazuje, że na końcu linii znajduje się strzałka zdefiniowana w szablonie. Modyfikacja linii znajduje się w kodzie 21.

Kod. 21. Modyfikacja krawędzi grafu

```
  <ng-template #linkTemplate let-link>
    <svg:g class="edge">
      <svg:path class="line" stroke-width="2" marker-end="url(#arrow)"> </svg:path>
    </svg:g>
  </ng-template>
```

Źródło: opracowanie własne

Rysownie klastrów odbywa się w podobny sposób jak czynności. Fragment kodu przedstawiający rysowanie toru pływackiego znajduje się w kodzie 22. Atrybut „stoke-width" określa szerokość linii obramowania prostokąta. „Stroke" oznacza kolor linii obramowania. „rx" odpowiada za zaokrąglenie narożników klastra.

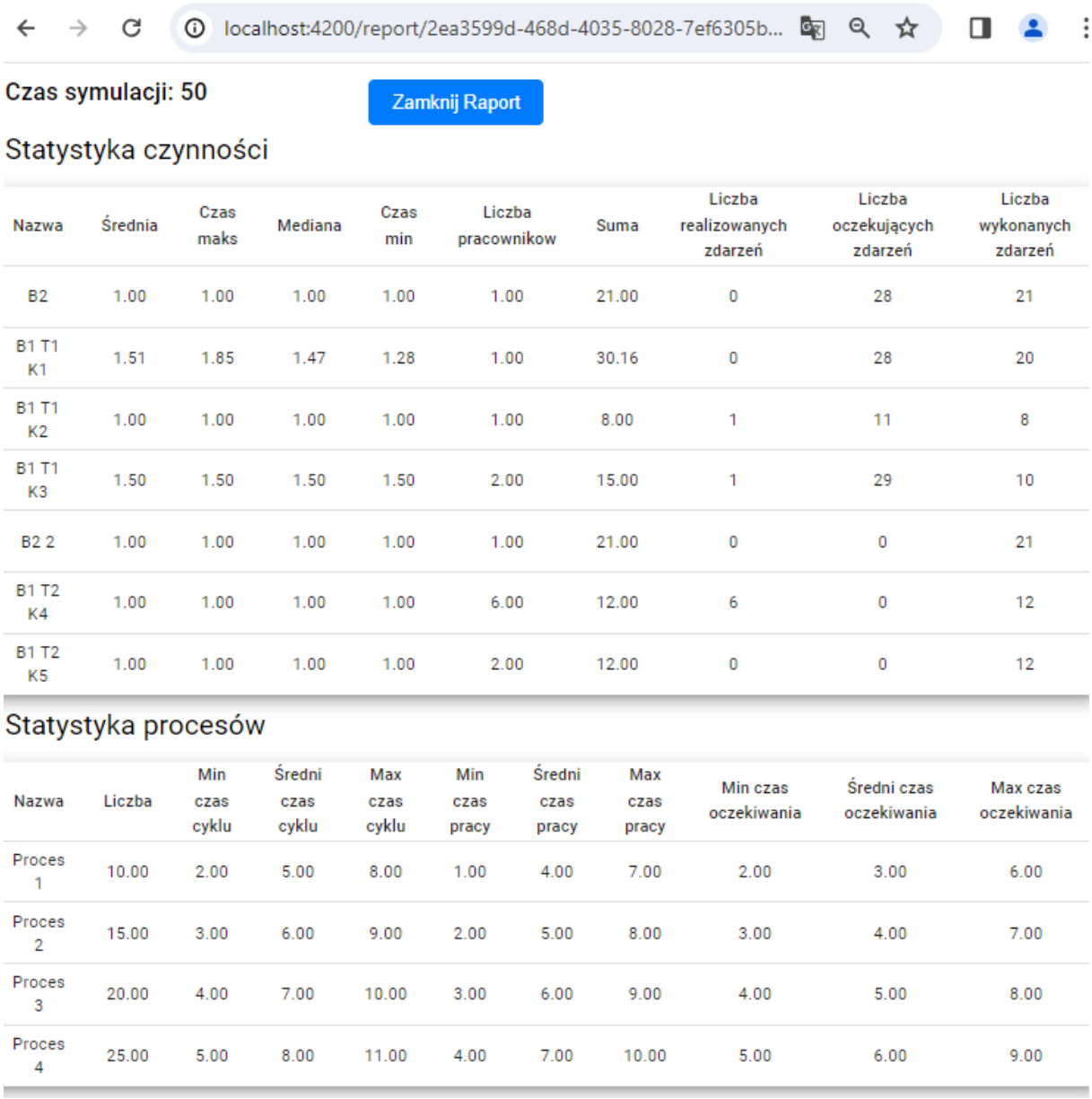
Kod. 22. Rysowanie toru pływackiego

```
<svg:rect
        [attr.width]="cluster.dimension.width"
        [attr.height]="cluster.dimension.height "
        [attr.stroke]="'#070708'"
        [attr.stroke-width]="3"
        [attr.fill]="'#0F7AF4'"
        [attr.rx]="cluster.data.borderRadius"
      />
<svg:rect
```

Źródło: opracowanie własne

III.3. Ekran raportu

Ekran raportu przedstawia informacje na temat symulacji. Na rysunku 15 pokazany jest interfejs z raportem. Interfejs zawiera informacje o czasie symulacji, przycisk zamykający raport i tabele przedstawiające szczegółowe informacje o procesach i czynnościach.



Rys. 15. Ekran raportu symulacji

Źródło: opracowanie własne

Po przejściu na stronę pobierane są informacje o procesach i czynnościach za pośrednictwem serwisu ReportService. Pobranie informacji o procesach symulacji przedstawione zostało w kodzie 23.

Kod. 23. Pobranie raportu o czynnościach

```
1    getRunningSimulation(id: string): Observable<any>{
2      return
   this.http.get<DataReport>(`http://localhost:8000/reports/${id}`)
   ;
3    }
```

Źródło: opracowanie własne

Na podstawie komponentu [7] tablica statystki czynności wyświetlana jest po pobraniu informacji. Fragment kodu przedstawiający kolumnę tabeli znajduje się w kodzie 24. W „th" zdefiniowany jest nagłówek kolumny tabeli. „td" zawiera wartość jaka zostanie wyświetlona w kolumnie (element.name). „*matCellDef" zawiera informacje, że każda komórka tabeli w kolumnie będzie wyświetlać informacje z listy raportu o czynnościach. Tablica ze statystykami procesów wyświetlana jest w podobny sposób.

Kod. 24. Kolumna tabeli

```
1           <ng-container matColumnDef="name">
2             <th mat-header-cell *matHeaderCellDef class="center-
    cell"> Nazwa </th>
3             <td mat-cell *matCellDef="let element" class="center-
    cell"> {{element.name}} </td>
4           </ng-container>
```

Źródło: opracowanie własne

Przycisk „Zamknij Raport" pozwala użytkownikowi na zamkniecie ekranu raportu i przenosi użytkownika do ekranu wyboru organizacji. Kod przycisku został przedstawiony w kodzie 25.

Kod. 25. Przycisk zamknięcia raportu

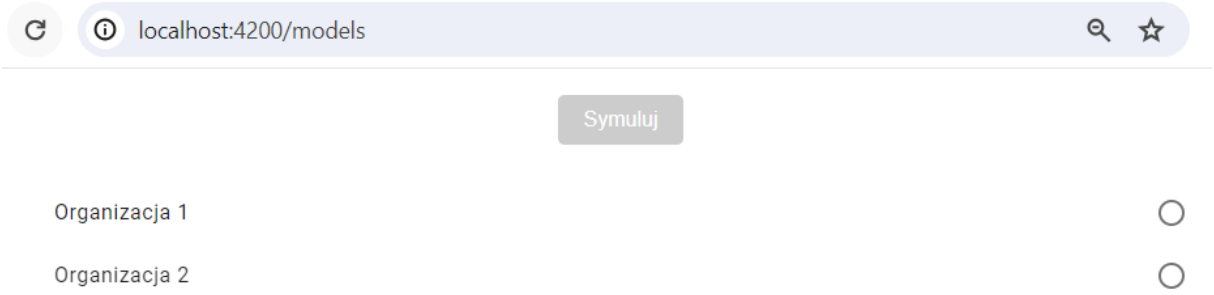
```
1   <button routerLink="/models" class="btn btn-primary custom-
    button">Zamknij Raport</button>
```

Źródło: opracowanie własne

Rozdział IV. Testy

Aplikacja jest zaprojektowana do wizualizacji symulacji procesów biznesowych. Dla celów testowych i sprawdzenia poprawności działania interfejsów użytkownika informacje na temat symulacji, które aplikacja otrzymywałaby z silnika symulacji zostały zamokowane. Według autora [4] są to fałszywe obiekty w systemie, które sprawdzą, czy testowany obiekt współdziała zgodnie z oczekiwaniami z fałszywym obiektem. Obiekty te naśladują rzeczywiste obiekty w kontrolowany sposób. Są używane do testowania i symulacji, przy czym pozwalają na odizolowanie komponentu, który jest testowany od zewnętrznych zależności, które mogą być trudne do kontrolowania lub symulowania. W przypadku aplikacji wizualizacji symulacji procesów biznesowych dzięki takiemu rozwiązaniu można przeprowadzić analizę czy czynności i bramki ułożone są we właściwej kolejności. Informacje użyte do testowania: organizacje, procesy biznesowe, czynności, zasoby, generatory, bramki XOR, bramki OR, bramki AND są zawarte w pliku „dane”.

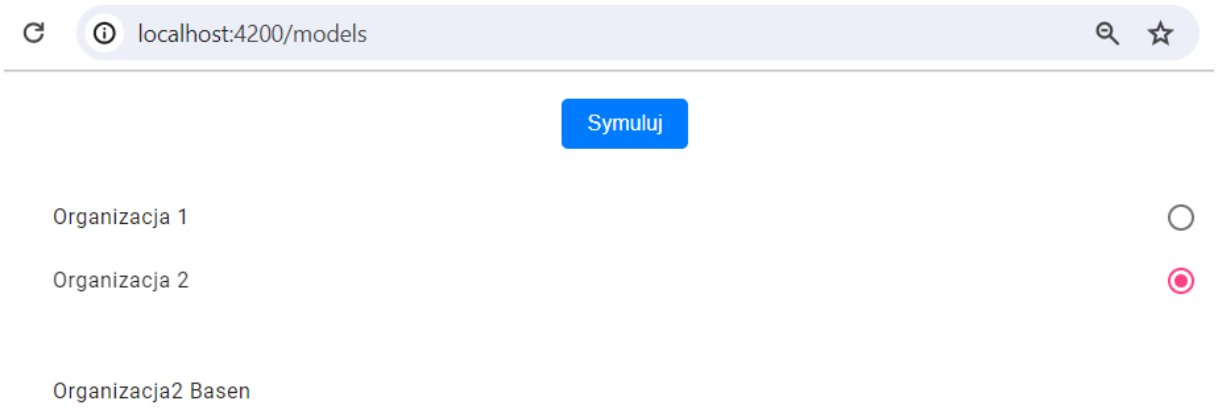
Z danych użytych do testowania wynika, że w liście organizacji powinny wyświetlić się dwie organizacje: „Organizacja 1” i „Organizacja 2”. Procesy biznesowe w „Organizacja 1” to: „Basen 1” i „Basen 2”, a dla „Organizacja 2” powinien się wyświetlić proces: „Organizacja2 Basen”. Na rysunku 16 przedstawiony został ekran wyboru organizacji. Organizacje przedstawione w liście są prawidłowe, dodatkowo przycisk jest szary. Żadna organizacja nie została wybrana, dlatego jest on nieaktywny.



Rys. 16. Ekran wyboru organizacji

Źródło: opracowanie własne

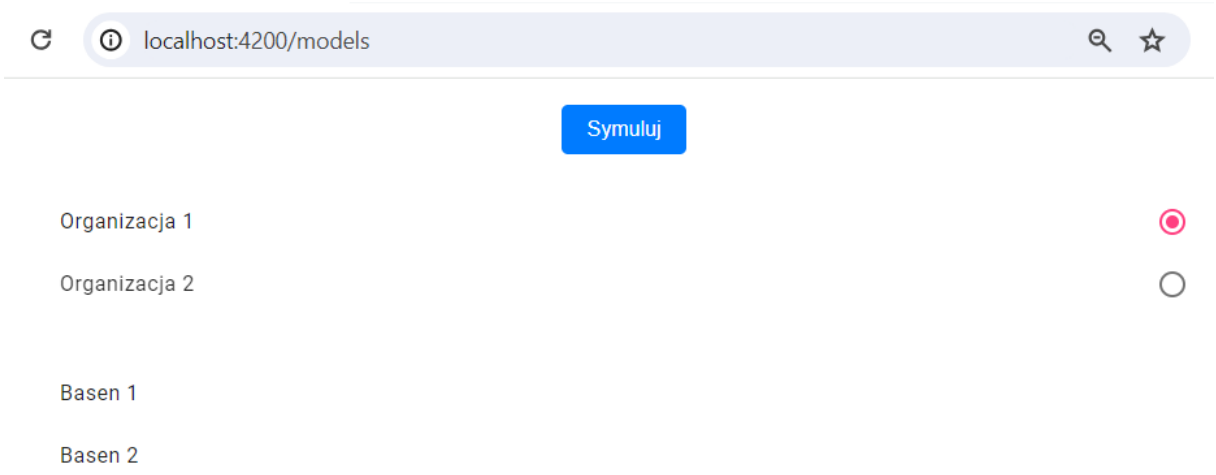
Interfejs użytkownika po wybraniu z listy jednokrotnego wyboru „Organizacja 2” znajduje się na rysunku 17. Wyświetlony został proces „Organizacja2 Basen” i przycisk zmienił kolor na niebieski. Przycisk „Symuluj” jest aktywny. Wyświetlony proces zgadza się z oczekiwaniami.



Rys. 17. Ekran wyboru organizacji po dokonaniu wyboru

Źródło: opracowanie własne

Interfejs użytkownika po dokonaniu zmiany na liście jednokrotnego wyboru z „Organizacja 2” na „Organizacja 1” przedstawiony został na rysunku 18. Wyświetlany proces zmienił się na „Basen 1” i „Basen 2”. Wynik zgadza się z oczekiwaniem.



Rys. 18. Ekran wyboru organizacji po zmianie organizacji na „Organizacja 1”

Źródło: opracowanie własne

Po naciśnięciu przycisku „Symuluj” następuje przejście do ekranu symulacji. W procesie „Basen 2” powinny znaleźć się dwa tory pływackie o nazwach: „Tor 1” i „Tor 2”. Pierwsza czynność powinna znajdować się w „Tor 1”, czynność powinna być zadaniem manualnym i następnym elementem powinna być bramka XOR, też w tym samym torze. Bramka powinna wskazywać na dwie czynności, obie w „Tor 2”, z czego jedna czynność powinna być zadaniem manualnym. Spis elementów, które znajdują się w procesie „Basen2” został przedstawiony w tabeli 1. W procesie „Basen 1” powinny znaleźć się trzy tory pływackie: „Tor 1”, „Tor 2” i „Tor 1”. W „Tor 3” powinny się znaleźć dwie czynności, w tym jedno zadanie manualne oraz bramki AND i OR. W torze „Tor 3 powinna być jedna czynność, a w ostatnim torze dwie czynności. Spis elementów, które znajdują się w procesie „Basen1” został przedstawiony w tabeli 2.

Tab. 1. Spis branek i czynności w procesie „Basen 2”

Typ elementu	Nazwa	Następny element	Poprzedni element	Zasób
Zadanie użytkownika	B2	Bramka XOR	Zdarzenie startu	Tor 1
Bramka XOR	X	B2 2, B2 3	B2	Tor 1
Zadanie manualne	B2 3	B2 2	Bramka XOR	Tor 2
Czynność	B2 2	Zdarzenie zakończenia	Bramka XOR	Tor 2

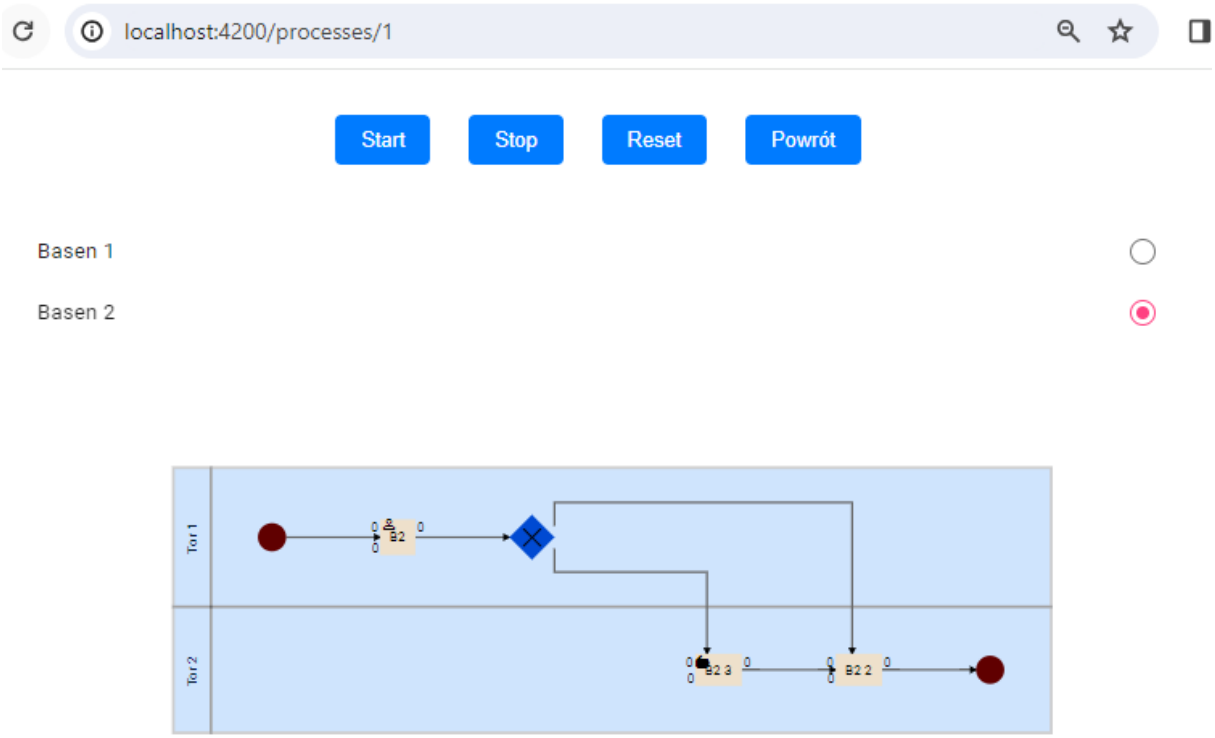
Źródło: opracowanie własne

Tab. 2. Spis bramek i czynności w procesie „Basen 1”

Typ elementu	Nazwa	Następny element	Poprzedni element	Zasób
Zadanie usługowe	B1 T1 K1	Bramka AND	Zdarzenie startu	Tor 1
Bramka AND	+	B1 T1 K2, B1 T1 K3	B1 T1 K1	Tor 1
Czynność	B1 T1 K2	Bramka OR	Bramka AND	Tor 1
Bramka OR	O	B1 T2 K4, B1 T2 K5	B1 T1 K2	Tor 2
Czynność	B1 T2 K5	Zdarzenie zakończenia	Bramka OR, B1 T2 K4	Tor 2
Czynność	B1 T2 K4	Bramka OR, B1 T1 K3	Bramka OR, B1 T1 K3	Tor 2
Czynność	B1 T1 K3	B1 T1 K4	Bramka AND	Tor 3

Źródło: opracowanie własne

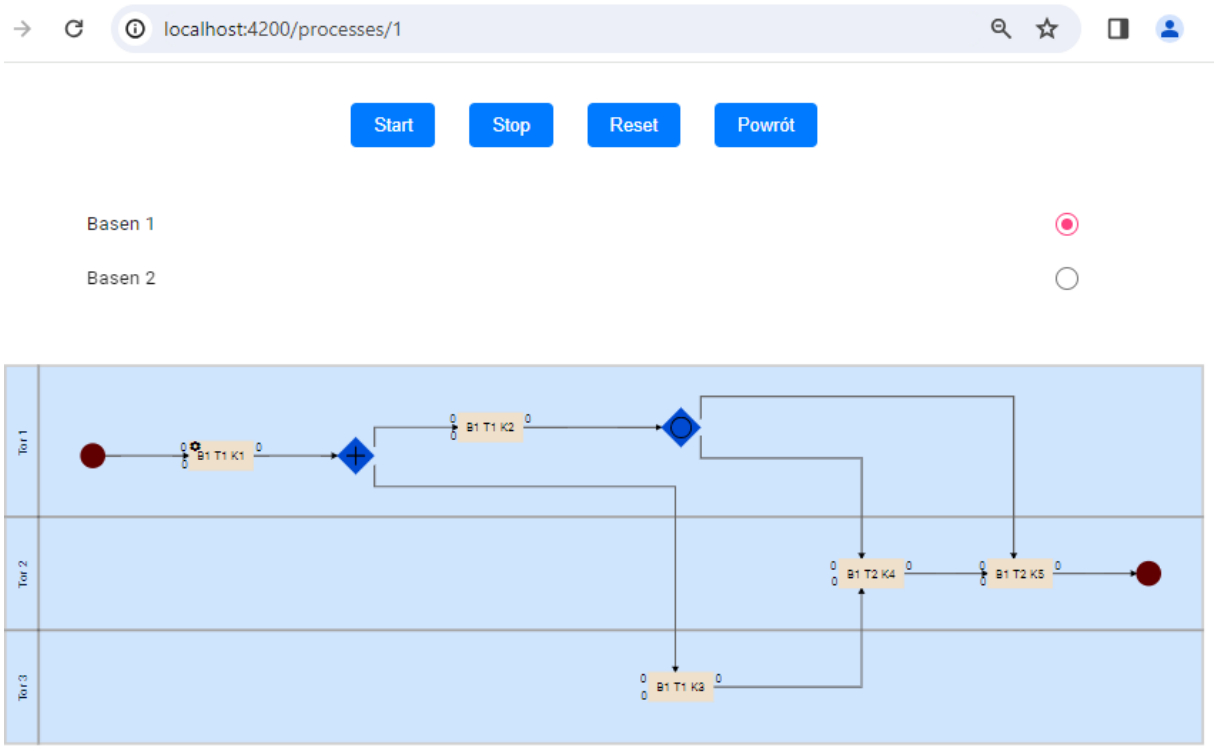
Rysunek 19 przedstawia proces „Basen 2”, czynności i bramki ułożone są w poprawnej kolejności, zgodnie z tabelą 1. Zadanie manualne i użytkownika również są pokazane. W ścieżce URL znajduje się parametr o wartości 1, co odpowiada odpowiedniemu identyfikatorowi wybranej organizacji we wcześniejszym widoku.



Rys. 19. Ekran symulacji po wybraniu procesu „Basen 2”

Źródło: opracowanie własne

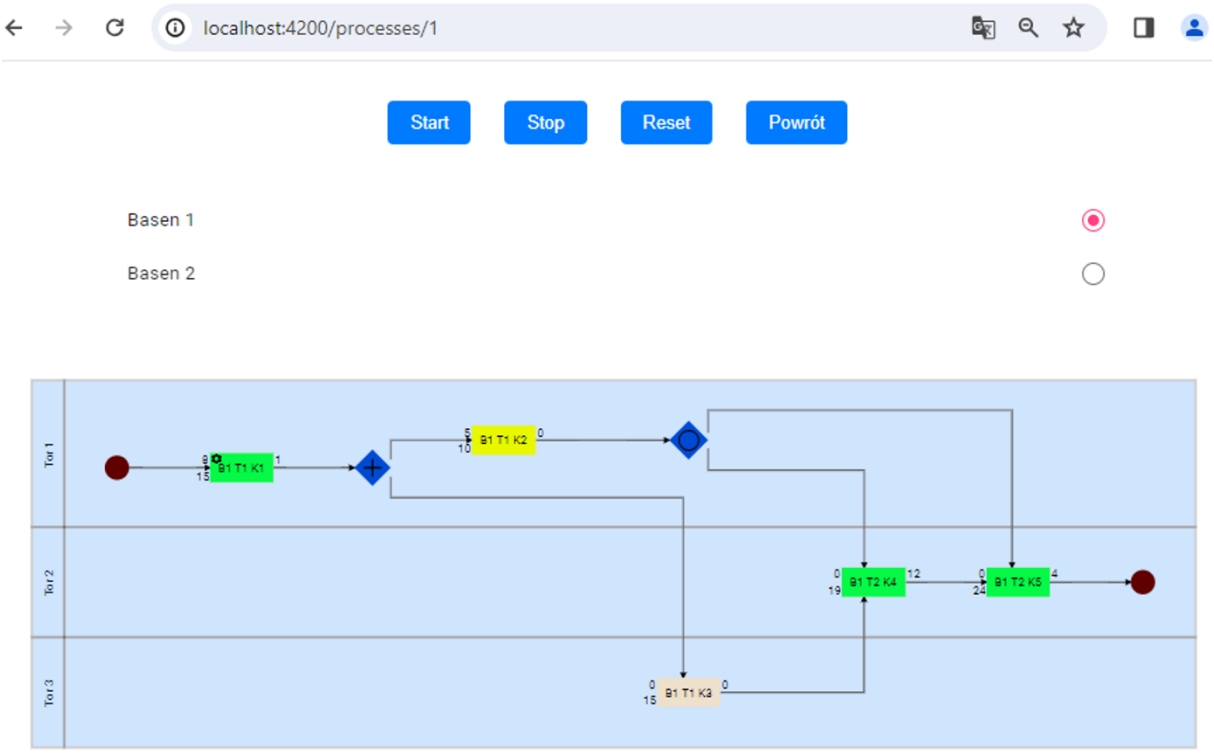
Po zmianie procesu na liście jednokrotnego wyboru na „Basen 1” nastąpiła zmiana wyświetlanego grafu. Interfejs użytkownika po zmianie procesu został przedstawiony na rysunku 20. W torze pływackim „Tor 1” znajduje się pierwsza czynność, która jest zadaniem manualnym, następna jest bramka AND, wskazująca na czynność w tym samym torze i czynność w torze „Tor 3”. Ułożenie elementów jest zgodne z tabelą 2.



Rys. 20. Ekran symulacji po zmianie procesu na „Basen 1”

Źródło: opracowanie własne

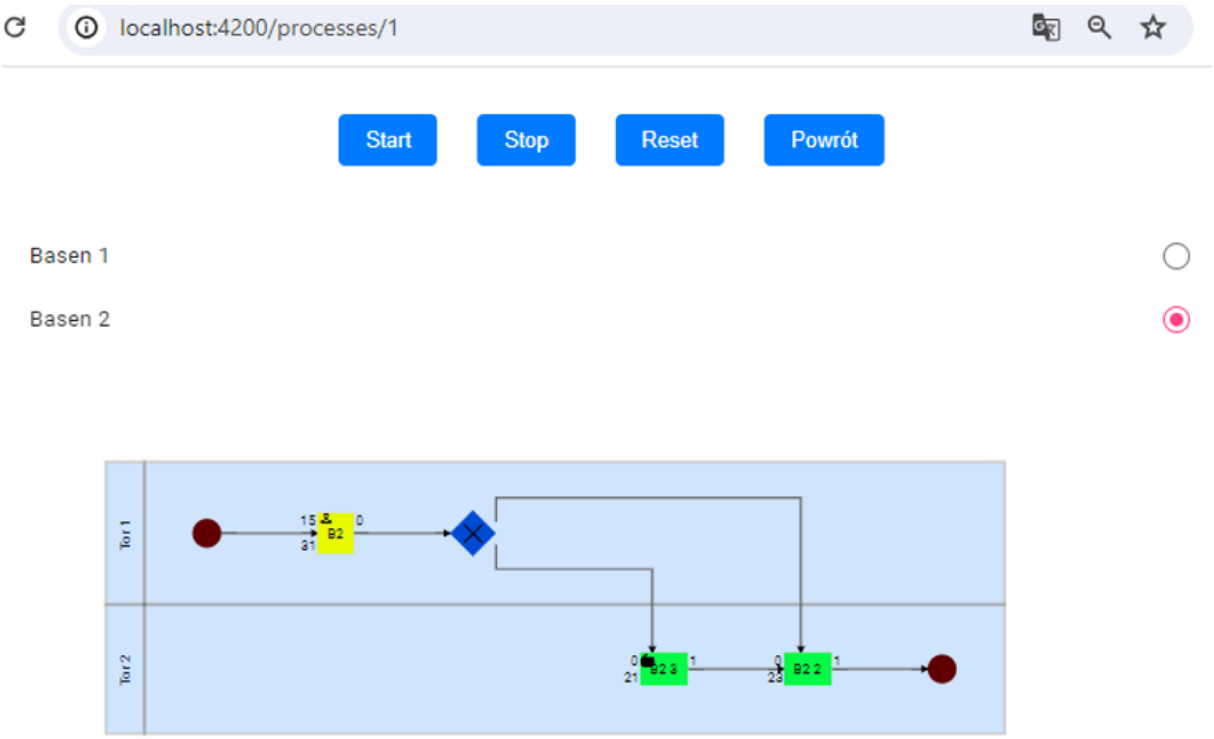
Po naciśnięciu przycisku „Start” powinna się rozpocząć symulacja, czyli liczniki procesów oczekujących, realizowanych i zrealizowanych, powinny być aktualizowane. Oczekiwanie procesu na zasób i realizowanie procesu powinny być widoczne poprzez zmiany koloru czynności. Interfejs użytkownika w trakcie symulacji znajduje się na rysunku 21. Liczniki i kolory czynności są aktualizowane. W „Tor 1” zadanie manualne jest na zielono – licznik procesów realizowanych w prawym górnym rogu procesu wskazuje, że jedna czynność jest realizowana. Druga czynność w tym torze świeci się na żółto, ponieważ pięć procesów oczekuje na realizację i żaden nie jest realizowany zatem czynność oczekuje na zasób. Czynności w pozostałych torach również są aktualizowane i następuje w nich zmiana koloru.



Rys. 21. Interfejs użytkownika w trakcie trwania symulacji

Źródło: opracowanie własne

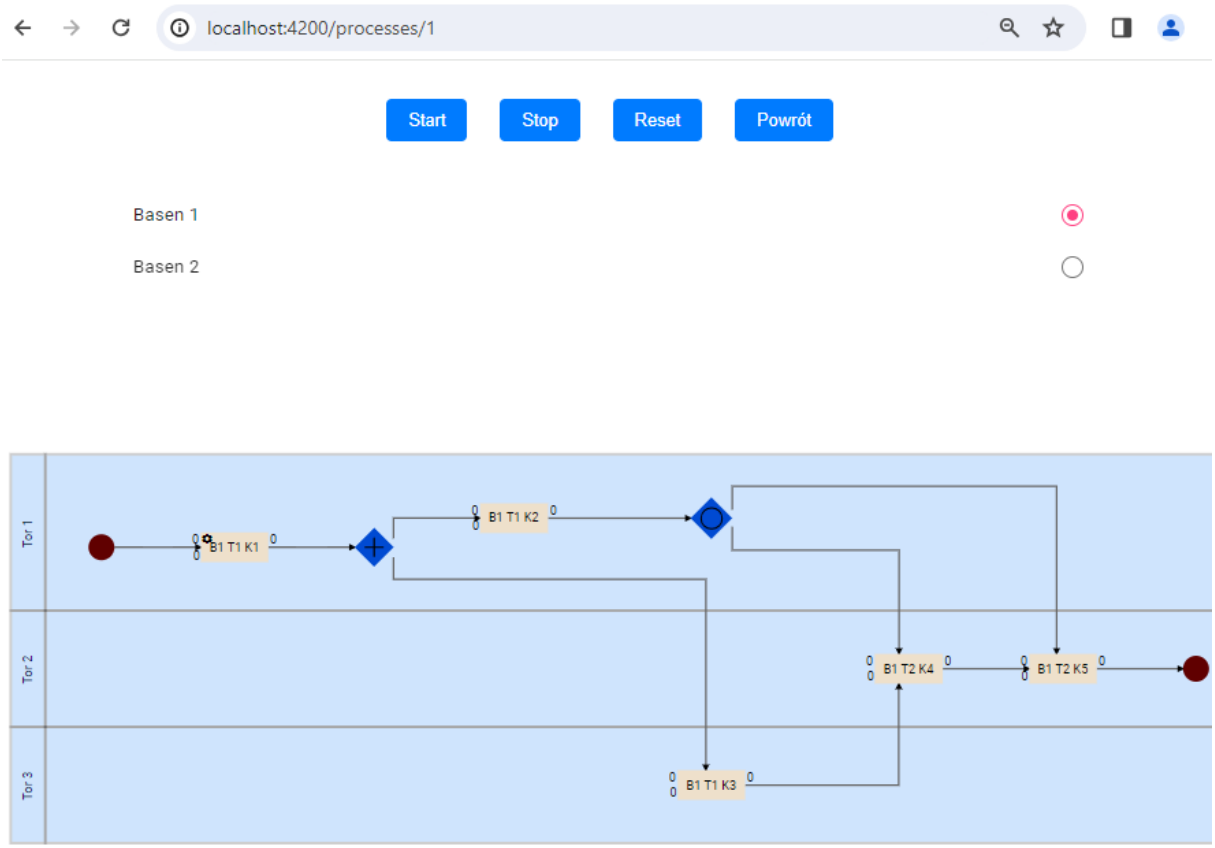
W trakcie symulacji użytkownik powinien mieć możliwość zamiany wyświetlanego procesu na inny, dostępny na liście jednokrotnego wyboru. Ekran po zmianie procesu na „Basen 2” w trakcie symulacji znajduje się na rysunku 22. W zadaniu użytkownika licznik procesów oczekujących wynosi 15, liczba procesów realizowanych to 0, zatem czynność ta zgodnie z oczekiwaniami jest koloru żółtego. Pozostałe dwie czynności w tym procesie są koloru zielonego, w obydwu przypadkach licznik reprezentujący liczbę procesów realizowanych wynosi 1.



Rys. 22. Interfejs użytkownika po zmianie procesu na „Basen 2” w trakcie trwania symulacji

Źródło: opracowanie własne

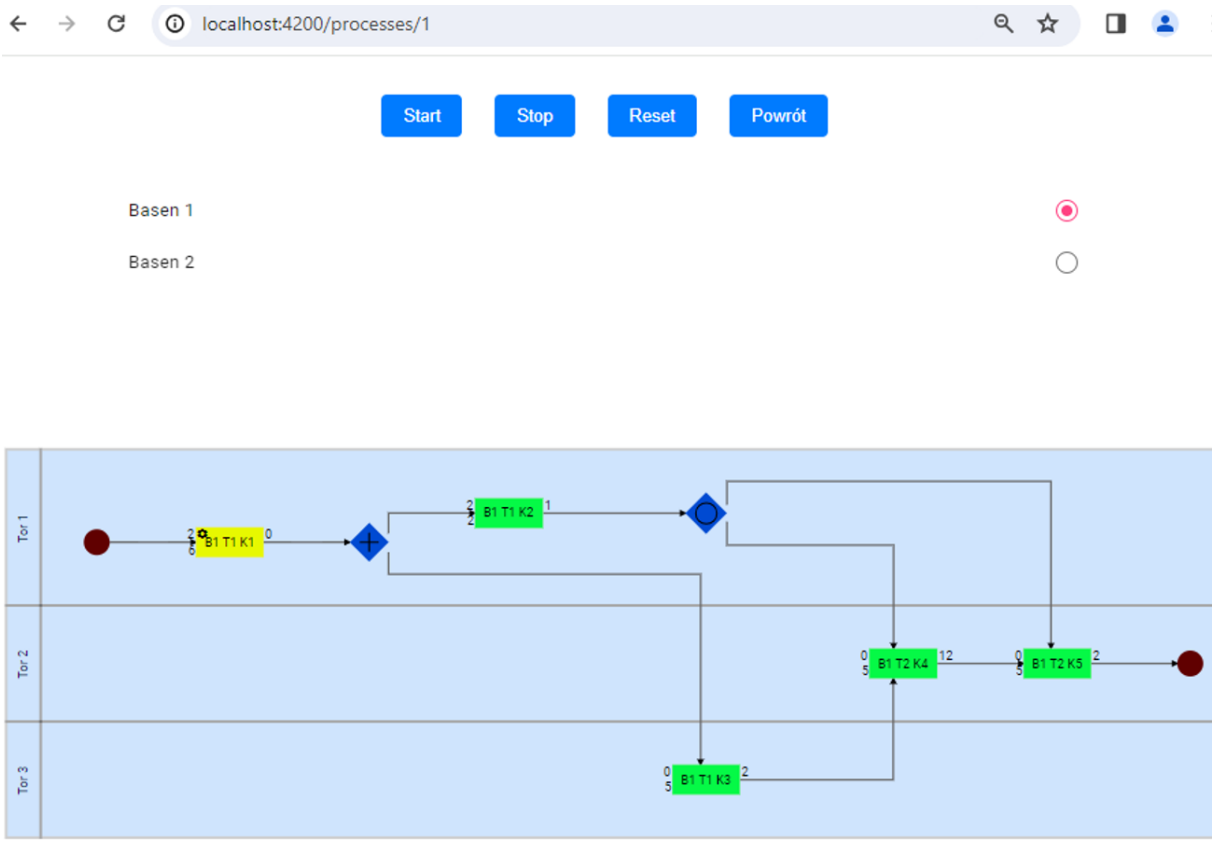
Po naciśnięciu przycisku „Reset” wszystkie procesy organizacji powinny wrócić do stanu początkowego. Na rysunku 23 przedstawiony jest „Basen 1”. W obu przypadkach procesy wróciły do stanu sprzed rozpoczęcia symulacji.



Rys. 23. Interfejs użytkownika po naciśnięciu przycisku „Reset”

Źródło: opracowanie własne

Po ponownym naciśnięciu przycisku „Start”, rozpoczyna się od nowa symulacja całej organizacji. Rysunek 24 przedstawia interfejs użytkownika w trakcie trwania nowej symulacji dla procesu „Basen 1”. Zadanie manualne oczekuje na zasoby zatem wyświetla się na żółto, pozostałe czynności są w trakcie realizacji, czyli świecą się na zielono.

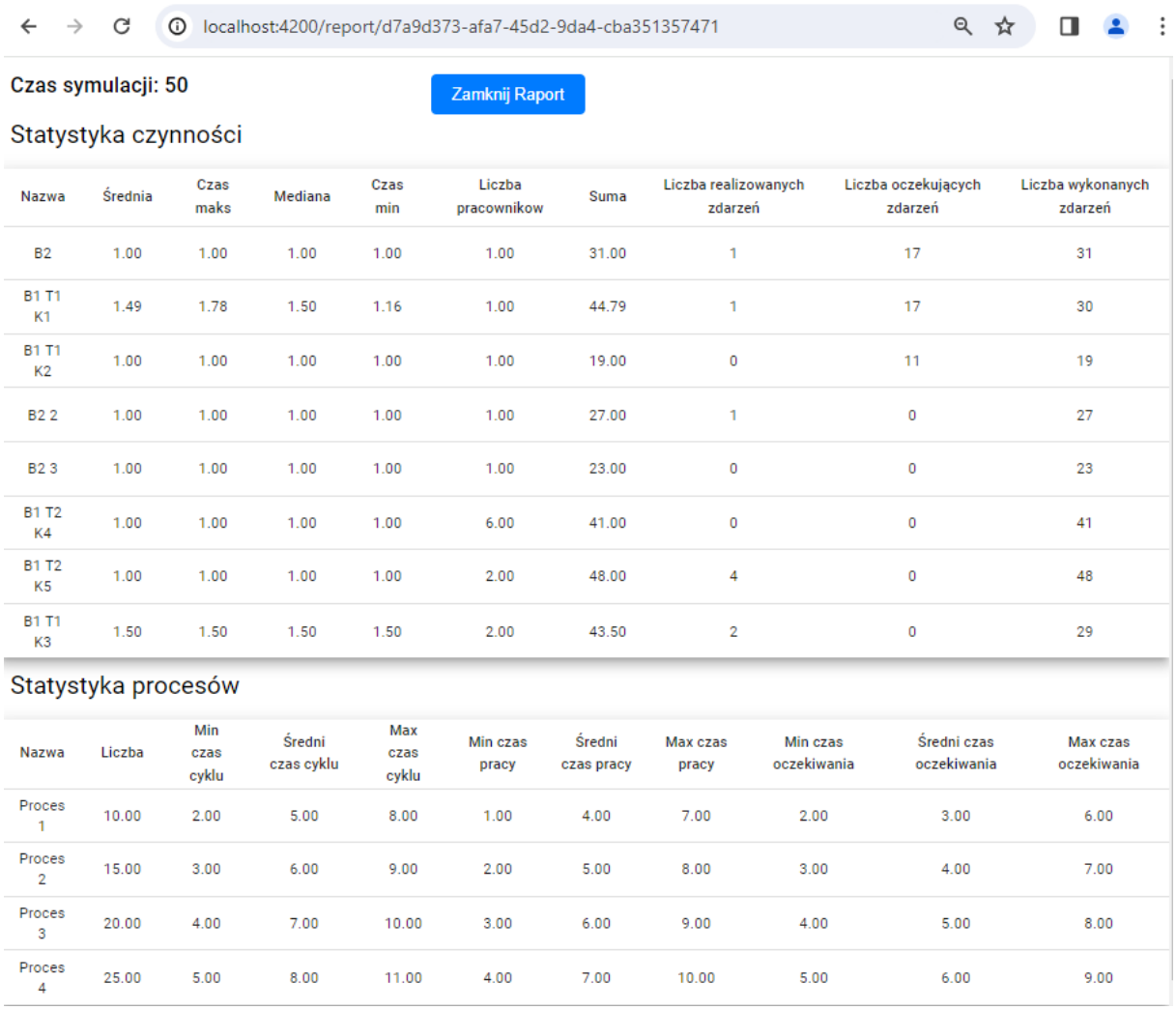


Rys. 24. Interfejs użytkownika po ponownym uruchomieniu symulacji

Źródło: opracowanie własne

Po zakończonej symulacji użytkownik zostaje przekierowany do ekranu raportu. Interfejs użytkownika z raportem znajduje się na rysunku 25. W ścieżce URL znajduje się d7a9d373-afa7-45d2-9da4-cba351357471 co jest identyfikatorem symulacji. W interfejsie użytkownika przedstawione są szczegółowe parametry symulacji dla procesów i czynności.

Testy dla zamokowanych danych, które są umieszczone w pliku „dane” wypadły pomyślne. Ekran wyboru organizacji prawidłowo wyświetlał organizacje, a po wybraniu organizacji wyświetlał procesy danej organizacji. Ekran symulacji przedstawiał procesy biznesowe zgodnie z oczekiwaniami. Po uruchomieniu symulacji liczniki czynności się aktualizowały i zmieniały się kolory czynności. Po zakończeniu symulacji użytkownik został przekierowany do raportu.



Rys. 25. Interfejs użytkownika z raportem symulacji

Podsumowanie

W pracy przedstawiono zagadnienia związane z symulacją procesów biznesowych, podkreślając jej znaczenie dla współczesnych przedsiębiorstw. Zbudowane środowisko wizualizacji pozwala na analizę procesów. Może ona pomóc w modernizacji lub poprawie działalności przedsiębiorstwa. Wizualizacja umożliwia zlokalizowanie słabych punktów w organizacji, których eliminacja może poprawić działanie firmy.

Przybliżone zostały zagadnienia związane z teoretycznymi podstawami symulacji procesów biznesowych, co pozwoliło na zrozumienie ich istoty i zastosowania w praktyce. Przedstawiono kluczowe pojęcia związane z procesami biznesowymi oraz ich symulacji. Wyjaśniło to korzyści jakie daje symulacja procesów biznesowych.

Kluczowym aspektem aplikacji jest jej techniczna struktura wraz z architekturą, która zapewnia podział obowiązków. Trudność polegała na przetworzeniu danych i odpowiednim ich wyświetleniu, dlatego model danych jest ważnym elementem w efektywnym przeprowadzaniu symulacji.

Opis implementacji aplikacji wyjaśnił funkcjonalności interfejsów użytkownika oraz interakcje możliwe do przeprowadzenia w środowisku symulacyjnym. Intuicyjne ekrany przekładają się na komfort użytkowania aplikacji.

Środowisko symulacyjne zostało zaprojektowane poprawnie, testy potwierdziły użyteczność aplikacji. Przejście przez scenariusz testowy pozwoliło na weryfikację działania programu i przybliżył funkcjonowanie aplikacji.

Zbudowane środowisko wizualizacji symulacji procesów biznesowych stanowi wartościowe narzędzie, które pozwala na lepsze zrozumienie działania przedsiębiorstw. Jego praktyczne zastosowanie może przyczynić się do poprawy efektywności operacyjnej i strategicznej, co jest kluczowe w dynamicznie zmieniającym się środowisku biznesowym.

Bibliografia

[1] Dumas M., La Rosa M., Mendling J, Reijers H. A.: Fundamentals of Buisness Process Management. Springer-Verlag Berlin Heidelberg, Heidelberg, Germany, 2013.
[2] Anupindi R., Chopra S., Deshmukh S. D., Van Mieghem J. A., Zemel E.: Managing Business Process Flows. Pearson Education, Inc., publishing as Prentice Hall, Upper Saddle River, New Jersey, 2012.
[3] Drejewicz S.: Zrozumieć BPMN MODELOWANIE PROCESÓW BIZNESOWYCH. Helion, Polska, 2012.
[4] Osherove R.: The Art of Unit Testing, Second Edition. Manning Publications, USA, 2014.
[5] https://swimlane.github.io/ngx-graph/#introduction [dostęp: 10.12.2023].
[6] https://angular.io/ [dostęp: 11.12.2023].
[7] https://material.angular.io/guide/schematics [dostęp: 12.12.2023].
[8] https://www.npmjs.com/package/d3-shape [dostęp: 13.12.2023].
[9] https://material.angular.io/components/icon/overview [dostęp: 13.12.2023].
[10] https://rxjs.dev/guide/overview [dostęp: 14.12.2023].

Spis kodów

Kod. 1. Lista jednokrotnego wyboru organizacji 22
Kod. 2. Warunek wyświetlania listy procesów organizacji 22
Kod. 3. Przycisk „Symuluj” 23
Kod. 4. Metody do pobierania danych o organizacjach i procesach 23
Kod. 5. Struktura programu ngx-graph 25
Kod. 6. Domyślne ustawienia grafu 27
Kod. 7. Funkcja dagereToOutput 27
Kod. 8. Funkcja dagereToOutput2 28
Kod. 9. Funkcja point 29
Kod. 10. Pobranie identyfikatora organizacji 29
Kod. 11. Pobranie informacji o procesach 29
Kod. 12. Aktualizacja liczników czynności 30
Kod. 13. Sprawdzenie statusu symulacji 30
Kod. 14. Przywrócenie liczników czynności do stanu początkowego 31
Kod. 15. Przycisk „Powrót” 31
Kod. 16. Modyfikacja wierzchołka grafu, będącego zdarzeniem startu lub zakończenia 32
Kod. 17. Modyfikacja wyglądu zadania użytkownika 32
Kod. 18. Wyświetlanie nazwy czynności 33
Kod. 19. Zmiana koloru czynności na zielony 33
Kod. 20. Zdefiniowanie znaku grotu 34
Kod. 21. Modyfikacja krawędzi grafu 34
Kod. 22. Rysowanie toru pływackiego 34
Kod. 23. Pobranie raportu o czynnościach 36
Kod. 24. Kolumna tabeli 36
Kod. 25. Przycisk zamknięcia raportu 36

Spis rysunków

Rys. 1. Zadanie usługowe 6
Rys. 2. Zadanie użytkownika 6
Rys. 3. Zadanie manualne 6
Rys. 4. Bramka XOR 7

Rys. 5. Bramka AND 7
Rys. 6. Bramka OR 8
Rys. 7. Diagram przypadków użycia 13
Rys. 8. Architektura MVVM 15
Rys. 9. Struktura aplikacji 17
Rys. 10. Model danych aplikacji 21
Rys. 11. Interfejs wyboru organizacji 22
Rys. 12. Interfejs użytkownika po wyborze organizacji 23
Rys. 13. Ekran symulacji przed wyborem procesu 24
Rys. 14. Ekran symulacji po wybraniu procesu 25
Rys. 15. Ekran raportu symulacji 35
Rys. 16. Ekran wyboru organizacji 37
Rys. 17. Ekran wyboru organizacji po dokonaniu wyboru 38
Rys. 18. Ekran wyboru organizacji po zmianie organizacji na „Organizacja 1” 38
Rys. 19. Ekran symulacji po wybraniu procesu „Basen 2” 40
Rys. 20. Ekran symulacji po zmianie procesu na „Basen 1” 41
Rys. 21. Interfejs użytkownika w trakcie trwania symulacji 42
Rys. 22. Interfejs użytkownika po zmianie procesu na „Basen 2” w trakcie trwania symulacji 43
Rys. 23. Interfejs użytkownika po naciśnięciu przycisku „Reset” 44
Rys. 24. Interfejs użytkownika po ponownym uruchomieniu symulacji 45
Rys. 25. Interfejs użytkownika z raportem symulacji 46

Spis tabel

Tab. 1. Spis branek i czynności w procesie „Basen 2” 39
Tab. 2. Spis bramek i czynności w procesie „Basen 1” 39

Załączniki

1. Płyta CD/DVD zawierająca:
    a. Prezentację wyników pracy dyplomowej
    b. Kody źródłowe oprogramowania
    c. Plik z danymi użytymi do testów